

Kubernetes for Application Developers

A 3-Day Study Book for the CKAD Exam — built on the podkit toolkit

Generated with podkit

September 2026

Contents

Read this first

Who this book is for

You want to pass the **CKAD** (Certified Kubernetes Application Developer) exam, and you have three focused days. You can type commands in a terminal and you have seen a container before. That is all you need. Everything else is explained from the ground up, in plain words, with small examples you can run on your own laptop.

The book is built around **podkit**, the deployment toolkit that ships with it. podkit generates real Kubernetes YAML (Deployments, Services, Secrets, NetworkPolicies, probes, ServiceAccounts...) from a short description, and every generated file is a *worked example* of something the exam asks you to write by hand. When a chapter says “in podkit”, it points at the file where that concept is used for real.

How the exam works (know your enemy)

Fact	Detail
Format	Hands-on. A browser-based terminal connected to real clusters. No multiple choice.
Time	2 hours
Tasks	Around 15–20, each worth a few percent; you may do them in any order
Passing score	66 %
Allowed docs	<code>kubernetes.io/docs</code> , <code>kubernetes.io/blog</code> , <code>helm.sh/docs</code> (opened in the exam browser)
Tools	<code>kubectl</code> (alias <code>k</code> is set up), <code>vim/nano</code> , <code>helm</code> , <code>docker/podman</code> on some tasks

Fact	Detail
Clusters	Several; every task tells you which context to use — always run the given kubectl config use-context first
Retake	One free retake is included; the score report tells you which domains were weak
Version	Kubernetes 1.3x (the exam follows the latest release closely) — check the current details when you register

The five domains and their weight

Domain	Weight	What it really means
Application Design and Build	20 %	images, Pods, Jobs, CronJobs, multi-container Pods, volumes
Application Deployment	20 %	Deployments, rolling updates, blue/green, canary, Helm, Kustomize
Application Observability and Maintenance	15 %	probes, logs, top, debugging, API deprecations
Application Environment, Configuration and Security	25 %	ConfigMaps, Secrets, requests/limits, quotas, ServiceAccounts, RBAC, SecurityContext, CRDs
Services and Networking	20 %	Services, Ingress, NetworkPolicies, troubleshooting access

The largest domain is *configuration and security*; the two 20 % domains are about building and deploying. Networking is where most people lose points, so Day 3 is devoted to it.

The 3-day plan

Day	Morning (3 h)	Afternoon (3 h)	Evening (1–2 h)
1 — Build & run	Ch. 1 kubectl survival kit, Ch. 2 workloads	Ch. 3 images, Ch. 4 multi-container Pods	Ch. 5 configuration, Day-1 exercises
2 — Deploy, observe, secure	Ch. 6 Deployments & strategies, Ch. 7 Helm & Kustomize	Ch. 8 observability & debugging	Ch. 9 security & RBAC, Day-2 exercises
3 — Network &	Ch. 10 Services &	Ch. 12 mock exam (2	Review mistakes,

Day	Morning (3 h)	Afternoon (3 h)	Evening (1–2 h)
rehearse	Ingress, Ch. 11 NetworkPolicy	h, timed)	cheat sheet, second speed drill

Rules that make the plan work:

1. **Type every command.** Reading is not practising. The exam tests your fingers.
2. **Time every exercise.** Most exam tasks should take 3–6 minutes.
3. **Use the docs the way you will in the exam** — open kubernetes.io/docs, search, copy a YAML skeleton, edit it. Do not memorise YAML; memorise *where it is*.
4. Keep the cheat sheet (Appendix A) open while you practise, then try without it on Day 3.

Set up your practice cluster (15 minutes)

You need a real cluster. The podkit toolkit gives you one in Docker with a single command:

```
cd podkit
pip install -r requirements.txt && make install      # the `podkit` command
podkit doctor                                       # kubectl, docker, kind,
helm present?                                       # one-node cluster,
podkit kind create
context "kind-podkit"
kubectl get nodes
```

No Docker? Any cluster works: minikube, k3d, a cloud cluster (podkit cluster login ...), or the free killer.sh simulator you get with the exam registration.

Set up the exam aliases now and use them for three days so they become muscle memory:

```
alias k=kubectl
export do="--dry-run=client -o yaml"      # k run x --image=nginx $do > pod.yaml
export now="--force --grace-period=0"    # k delete pod x $now
source <(kubectl completion bash); complete -o default -F __start_kubectl k
```

Put these three lines in ~/.bashrc. In the exam the k alias and completion are usually already there; \$do and \$now you set yourself in ten seconds.

Vim settings that save minutes (~/.vimrc):

```
set tabstop=2 shiftwidth=2 expandtab
set number
```

How each chapter is organised

- **Plain words** — the idea, with an everyday comparison.
- **Step by step** — commands you run, and what you should see.
- **In podkit** — where the same thing is used in the toolkit's real code.
- **Exam tips** — the fast way, the traps, the docs page to open.
- **Exercises** — timed tasks in the exam's style, with solutions at the end of the day.

Words you will see a lot:

Word	Meaning
cluster	a group of machines run as one big computer
node	one of those machines (a VM, a laptop, a Raspberry Pi...)
control plane	the “brain” that decides where things run (API server, scheduler, controllers, etcd)
Pod	the smallest thing Kubernetes runs: one or more containers that share a network address and can share disk
manifest	a YAML file that describes a Kubernetes object
namespace	a folder that keeps objects apart (dev, prod, team-a)
label	a key=value sticker on an object; selectors pick objects by their stickers
controller	a program in the control plane that watches objects and makes reality match the spec

Let's start.

Day 1 — Build and run

Chapter 1 — The kubectl survival kit

Plain words

Kubernetes is a **manager of containers**. You tell it *what* you want (“three copies of my web app, listening on port 80”) and it works out *how*: which node has room, what to do when a container crashes, how traffic finds the app. You talk to it through one program: kubectl (say “cube-control” or “cube-C-T-L”).

Think of the cluster as a **school**. The control plane is the office: it keeps the list of students (etcd), takes your requests (the API server), and assigns kids to classrooms (the scheduler). Nodes are classrooms. Pods are the students — each one carrying a lunchbox (the containers). Controllers are teachers who count heads and fetch a replacement if a student goes missing.

Every object you create is a YAML document with the same four parts:

```
apiVersion: v1           # which "edition" of the API describes this kind
kind: Pod                 # what it is
metadata:                 # name, namespace, labels, annotations
  name: hello
spec:                     # what you want
  containers:
  - name: web
    image: nginx:1.27
```

status: is the fifth part; Kubernetes writes it, you read it.

Step by step: your first ten minutes with a cluster

```
kubectl config get-contexts      # which clusters do I know?
kubectl config use-context kind-podkit # switch (the exam tells you the context for every task!)
kubectl config set-context --current --namespace=default # default namespace for this context
kubectl get nodes -o wide
kubectl get all                  # Pods, Services, Deployments,
ReplicaSets in the namespace
kubectl get pods -A             # -A = all namespaces
kubectl get pods -n kube-system # the cluster's own Pods
```

The imperative commands (your best friends)

kubectl can create most objects from a one-liner. In the exam this is 3× faster than writing YAML from scratch.

```
kubectl run web --image=nginx:1.27 --port=80           # a Pod
kubectl create deployment web --image=nginx:1.27 --replicas=3 # a Deployment
kubectl expose deployment web --port=80 --target-port=80 # a Service
kubectl create configmap app-cfg --from-literal=MODE=prod # a ConfigMap
kubectl create secret generic db --from-literal=password=s3cr3t # a Secret
kubectl create job once --image=busybox -- echo hello      # a Job
kubectl create cronjob tick --image=busybox --schedule="*/5 * * * *" -- date
kubectl create serviceaccount app
kubectl create namespace team-a
```

The trick that wins the exam: `--dry-run=client -o yaml`

Instead of remembering YAML, **ask kubectl to write it**, then edit:

```
kubectl run web --image=nginx:1.27 --port=80 --dry-run=client -o yaml > pod.yaml
vim pod.yaml           # add what the task wants (env, volumes, probes...)
kubectl apply -f pod.yaml
```

With the alias from the front matter: `k run web --image=nginx $do > pod.yaml`.

Reading and changing objects

```
kubectl describe pod web           # human-readable, includes Events at the
bottom (read them!)
kubectl get pod web -o yaml         # the full object, as YAML
kubectl get pod web -o jsonpath='{.status.podIP}'
kubectl get pods -o wide            # IPs and nodes
kubectl get pods --show-labels
kubectl get pods -l app=web         # filter by label
kubectl get pods --field-selector status.phase=Running
kubectl edit deployment web         # opens vim on the live object; saving
applies it
kubectl label pod web tier=front     # add a label (--overwrite to change)
kubectl annotate pod web note="hi"
kubectl delete pod web              # add $now (--force --grace-period=0) to
skip the 30 s wait
kubectl delete -f pod.yaml
```

apply vs create vs replace

Command	Use when
<code>kubectl create -f x.yaml</code>	the object must not exist yet
<code>kubectl apply -f x.yaml</code>	create or update ; the normal way (works for repeated runs)
<code>kubectl replace -f x.yaml</code>	replace the whole object; <code>--force</code> deletes and recreates (needed to change immutable fields such as a Pod's image name in some cases, or a Job's template)

The built-in manual: `kubectl explain`

```
kubectl explain pod.spec.containers.livenessProbe
kubectl explain deployment.spec.strategy --recursive
kubectl api-resources           # every kind, its short name, apiVersion and
whether it is namespaced
kubectl api-versions
```

`explain` works offline and tells you the exact field names and types. When you are unsure whether a field is `initialDelaySeconds` or `initialDelay`, `explain` is faster than the docs.

In podkit

- The generated `build/<cloud>/<name>/k8s/workload.yaml` is exactly what `kubectl create deployment ... --dry-run=client -o yaml` would give you, with everything a

production Pod needs added (labels, probes, resources, security context). Open one and read it top to bottom.

- podkit pod deploy applies manifests with `kubectl apply --server-side --field-manager=podkit`. Server-side apply lets several tools manage one object without fighting; the exam only needs plain apply.
- Every podkit command accepts `--dry-run` for the same reason kubectl does: see what would happen before it happens.

Namespaces, labels and selectors in 5 minutes

Namespaces keep things apart. Most objects live in one (Pod, Service, Deployment, ConfigMap); a few are cluster-wide (Node, Namespace, PersistentVolume, ClusterRole).

```
kubectl create namespace dev
kubectl run web --image=nginx -n dev
kubectl get pods -n dev
kubectl config set-context --current --namespace=dev      # stop typing -n
```

Labels are stickers; selectors match stickers. A Deployment finds its Pods by labels, a Service sends traffic to Pods by labels, a NetworkPolicy allows Pods by labels. When a task says “the Service does not reach the Pods”, the answer is almost always **a label mismatch**.

```
kubectl get pods -l 'app=web,tier!=cache'
kubectl get pods -l 'env in (dev,test)'
kubectl get svc web -o jsonpath='{.spec.selector}'
kubectl get pods -l "$(kubectl get svc web -o jsonpath='{.spec.selector}' | tr
-d '{}' | tr : =)"
```

Exam tips

- The first line of every task is the context. `kubectl config use-context <name>` — do it every time, even when it looks the same.
- Read the task twice. Note the namespace, the name, the image with its **exact tag**.
- Create with an imperative command, then edit or dump YAML and fix. Never type YAML from memory when kubectl can draft it.
- When you are stuck for more than 6–8 minutes: write down the task number, move on, come back. Every task is worth points; an unfinished hard task costs the same as an easy one.
- `kubectl get events -A --sort-by=.lastTimestamp | tail` shows what just went wrong anywhere.
- Vim: `:set paste` before pasting YAML from the docs; `dd/p` to move lines; `>>/<<` to indent.

Exercises (20 minutes)

1. Create namespace `ckad-1`. In it, create a Pod named `hello` with image `busybox:1.36` that runs `sleep 3600`. Show its IP address with a single command.
2. From a dry-run, write the YAML for a Deployment `api` (image `nginx:1.27`, 2 replicas) to `api.yaml`, add the label `tier: backend` to the Pod template, and apply it. Confirm both Pods carry the label.

3. Which resources in the cluster are **not** namespaced? Find the answer with one command.
4. Without opening a browser, find the field that sets a Pod's DNS policy.

(Solutions at the end of Day 1.)

Chapter 2 — Pods and the workloads that manage them

Plain words

A **Pod** is the smallest thing you can run. It usually holds one container, sometimes a few that must live together. Pods are *mortal*: when the node dies, the Pod is gone and nobody recreates it. That is why you almost never create bare Pods for real work. Instead you create a **workload resource** — a controller that keeps Pods alive:

You want...	Use	The rule of thumb
a long-running app (web, API), rolling updates	Deployment	“keep N copies running, replace them one at a time when I change something”
exactly one Pod per node (log collector, monitoring agent)	DaemonSet	“one on every node, including new ones”
a task that runs once and finishes (migration, batch)	Job	“run to completion, retry on failure”
a task on a schedule (backup at 02:00)	CronJob	“a Job every * * * * *”
stable names and storage per replica (databases)	StatefulSet	“pod-0, pod-1, each with its own disk”
a Deployment’s Pods (never used directly)	ReplicaSet	created <i>by</i> the Deployment

The exam domain is called “choose and use the right workload resource”. Learn the table.

Step by step: the Pod

```
kubectl run web --image=nginx:1.27 --port=80 --labels=app=web,tier=front
kubectl get pod web -o wide           # READY 1/1, STATUS Running, its IP and
node
kubectl describe pod web              # look at Containers and Events
kubectl logs web                     # what the container printed
kubectl exec -it web -- sh           # a shell inside; exit with Ctrl-D
kubectl delete pod web $now
```

A Pod with command and arguments, environment and resource requests, from a dry run:

```
kubectl run worker --image=busybox:1.36 --restart=Never --env=MODE=fast \
--dry-run=client -o yaml -- sh -c 'echo working; sleep 3600' > worker.yaml
vim worker.yaml      # add the resources block shown below (kubectl explain
pod.spec.containers.resources)
```

The important lines in the result:

```
spec:
  containers:
  - name: worker
```

```

image: busybox:1.36
command: ["sh", "-c"]           # replaces the image's ENTRYPOINT
args: ["echo working; sleep 3600"] # replaces the image's CMD
env:
  - name: MODE
    value: fast
resources:
  requests: { cpu: 100m, memory: 64Mi }
  limits:   { cpu: 200m, memory: 128Mi }
restartPolicy: Never           # Always (default, for servers) |
OnFailure | Never

```

command = Docker's ENTRYPOINT, args = Docker's CMD. If a task says "the container must run /bin/app --verbose", write command: ["/bin/app"] and args: ["--verbose"], or put both in command.

Pod phases and what they mean

STATUS	Meaning	First thing to check
Pending	not scheduled or image not pulled yet	describe → Events (no node with enough CPU? PVC missing?)
ContainerCreating	starting	volumes/ConfigMaps missing?
Running	at least one container runs	READY column: 0/1 means a readiness probe fails
CrashLoopBackOff	the process keeps exiting	logs --previous
ImagePullBackOff / ErrImagePull	wrong image name/tag or private registry	describe → Events; typo in the tag
Completed	Job/one-shot Pod finished	fine
Error	exited non-zero	logs

Step by step: Deployment and ReplicaSet

```

kubectl create deployment web --image=nginx:1.27 --replicas=3 --port=80
kubectl get deploy,rs,pods -l app=web
kubectl scale deployment web --replicas=5
kubectl get pods -w           # watch the new Pods appear; Ctrl-C
kubectl delete pod -l app=web --wait=false # kill them all; the ReplicaSet
makes new ones

```

The Deployment owns a ReplicaSet; the ReplicaSet owns the Pods. When you change the Pod template (image, env, labels), the Deployment creates a **new** ReplicaSet and shifts Pods over (Chapter 6).

The generated YAML, trimmed:

```

apiVersion: apps/v1
kind: Deployment

```

```

metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }      # MUST match template labels
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports: [{ containerPort: 80 }]

```

Trap: spec.selector is immutable and must match spec.template.metadata.labels. If you add a label to the template that the selector does not have, that is fine; if you change a label the selector uses, the apply fails (“field is immutable”) — delete and recreate.

Step by step: Job and CronJob

```

kubectl create job pi --image=perl:5.34 -- perl -Mbignum=bpi -wle 'print
bpi(200)'
kubectl get jobs; kubectl get pods -l job-name=pi
kubectl logs job/pi

```

Job knobs you must know (all under spec:):

```

apiVersion: batch/v1
kind: Job
metadata: { name: pi }
spec:
  completions: 5           # run 5 successful Pods in total
  parallelism: 2           # at most 2 at the same time
  backoffLimit: 3          # give up after 3 failed Pods (default 6)
  activeDeadlineSeconds: 120 # kill the whole Job after 2 minutes
  ttlSecondsAfterFinished: 60 # delete the Job 60 s after it finishes
  template:
    spec:
      restartPolicy: Never  # Job Pods must be Never or OnFailure (NOT
Always)
      containers:
        - name: pi
          image: perl:5.34
          command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(200)"]

```

CronJob = a schedule that stamps out Jobs:

```

kubectl create cronjob backup --image=busybox:1.36 --schedule="0 2 * * *" -- sh
-c 'echo backup'
kubectl get cronjob backup
kubectl create job manual-run --from=cronjob/backup      # run it right now
apiVersion: batch/v1
kind: CronJob
metadata: { name: backup }
spec:
  schedule: "0 2 * * *"      # min hour day month weekday (crontab.guru)

```

```

concurrencyPolicy: Forbid           # Allow (default) | Forbid | Replace
successfulJobsHistoryLimit: 3
failedJobsHistoryLimit: 1
startingDeadlineSeconds: 200       # if it could not start within 200 s, skip
this run
jobTemplate:
  spec:
    backoffLimit: 2
    template:
      spec:
        restartPolicy: OnFailure
        containers:
          - name: backup
            image: busybox:1.36
            command: ["sh", "-c", "echo backup"]

```

Cron cheat sheet: */5 * * * * every 5 minutes · 0 * * * * hourly · 0 2 * * * daily 02:00 · 0 2 * * 1 Mondays 02:00 · 30 8 1 * * * the 1st of each month 08:30.

Step by step: DaemonSet and StatefulSet (know them, rarely write them)

There is no `kubectl create daemonset`; take a Deployment dry-run, change `kind: DaemonSet` and delete replicas and strategy:

```

kubectl create deployment log-agent --image=fluent/fluent-bit:3.0 $do \
| sed 's/kind: Deployment/kind: DaemonSet/' | grep -v 'replicas:' > ds.yaml

```

A StatefulSet needs a headless Service (`clusterIP: None`) named in `serviceName`, gives Pods ordered names (`db-0`, `db-1`) and a `volumeClaimTemplates` disk each. You will recognise it; you are unlikely to write one from scratch on the exam.

Scheduling basics (where does the Pod land?)

```

spec:
  nodeSelector:
    disktype: ssd                    # simplest: node must carry this label
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "batch"
      effect: "NoSchedule"          # allow scheduling on a tainted node
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - { key: kubernetes.io/os, operator: In, values: [linux] }

```

```

kubectl label node kind-podkit-control-plane disktype=ssd
kubectl taint node worker-1 dedicated=batch:NoSchedule
kubectl taint node worker-1 dedicated-                                # remove

```

In podkit

- `podgen/k8s.py --deployment()` builds the Deployment above with `revisionHistoryLimit: 2`, a RollingUpdate strategy (`maxSurge: 1`, `maxUnavailable: 0`) and the same selector/template labels (`app.kubernetes.io/name`).

- `scheduling.spot: true` in a `PodConfig` adds the cloud's node selector and toleration (podgen/config.py, table SP0T) — a real use of `nodeSelector` + tolerations.
- The command `pod` (`command-pod/k8s/command-pod.yaml`) is a `Deployment` whose only job is to `sleep infinity` so you can `exec` into it — a common exam pattern for “a Pod I can run commands from”.
- `restartPolicy` matters: the Java client/server examples are `Deployments` (`Always`); a one-shot migration would be a `Job` (`OnFailure`).

Exam tips

- `kubectl run` makes a **Pod**. `kubectl create deployment` makes a **Deployment**. Read which one the task wants.
- For a Pod that should run a command: `kubectl run x --image=busybox --restart=Never -- sh -c '...'` — the `--` separates `kubectl` flags from the container command.
- A Job's Pod template needs `restartPolicy: Never|OnFailure`; the imperative `create job` sets it for you.
- “The Job must finish within 30 seconds” → `activeDeadlineSeconds: 30`. “Retry at most twice” → `backoffLimit: 2`. “Run 3 Pods, 1 at a time” → `completions: 3, parallelism: 1`.
- Docs pages to bookmark: *Workloads — Pods, Jobs, CronJob, Deployments*.

Exercises (25 minutes)

1. Create a Pod `sleeper` (image `busybox:1.36`) that runs `sleep 300` and restarts only on failure. Verify with `kubectl get pod sleeper -o jsonpath='{.spec.restartPolicy}'`.
2. Create Deployment `store` with image `nginx:1.26`, 4 replicas, in namespace `shop` (create the namespace). Then scale it to 2 with a single command.
3. Create a Job `countdown` that prints the numbers 5 to 1 (`for i in 5 4 3 2 1; do echo $i; done`) using `busybox:1.36`, runs 3 times in total, 3 at once, and is cleaned up 30 seconds after it finishes. Show the logs of one of its Pods.
4. Create a CronJob `ping` that runs `date` every 10 minutes with image `busybox:1.36`, never allows two runs at the same time, and keeps only 1 successful Job. Trigger one run manually.
5. Turn the `store` Deployment's YAML into a `DaemonSet` named `store-ds` in namespace `shop`.

Chapter 3 — Container images: define, build, modify

Plain words

A **container image** is a frozen, shippable copy of a program and everything it needs (libraries, config, a tiny operating system). A **Dockerfile** is the recipe. A **registry** (Docker Hub, GHCR, ECR, ACR, GAR) is the shelf where images are stored, and a **tag** is the version sticker (nginx:1.27). Kubernetes nodes pull images from registries when a Pod starts.

The CKAD domain says “define, build and modify container images”. In practice a task gives you a Dockerfile (or asks for a tiny change), and you must build the image with docker or podman, tag it, and maybe save it to a tar file or push it.

Step by step: build, tag, save

A minimal recipe (the podkit Java example is a real one — see below):

```
FROM python:3.12-slim          # start from an official base image
WORKDIR /app                   # cd inside the image
COPY app.py .                  # copy from your machine into the image
RUN pip install --no-cache-dir flask # run at BUILD time
ENV PORT=8080                  # environment inside the container
EXPOSE 8080                    # documentation of the port (does not
publish it)
USER 1000                      # run as a non-root user
CMD ["python", "app.py"]       # run at START time (overridden by Pod args)
```

ENTRYPOINT vs CMD: ENTRYPOINT is the program, CMD its default arguments. In a Pod, command: replaces ENTRYPOINT and args: replaces CMD.

```
docker build -t myapp:1.0 .          # build from the Dockerfile in the
current directory
docker build -t myapp:1.0 -f docker/Dockerfile.prod .
docker images | grep myapp
docker run --rm -p 8080:8080 myapp:1.0 # quick local test
docker tag myapp:1.0 registry.example.com/team/myapp:1.0
docker push registry.example.com/team/myapp:1.0
docker save -o myapp-1.0.tar myapp:1.0 # export to a tarball (a classic
exam task)
docker load -i myapp-1.0.tar
docker image inspect myapp:1.0 --format '{{.Config.Cmd}}'
```

podman uses exactly the same words: podman build, podman tag, podman save, podman push. If the task says podman, type podman.

Load an image into your practice cluster (kind cannot see your local Docker images otherwise):

```
podkit kind load myapp:1.0          # = kind load docker-image myapp:1.0 --name
podkit
```

Using the image in a Pod

```
containers:
- name: app
  image: registry.example.com/team/myapp:1.0
  imagePullPolicy: IfNotPresent # Always | IfNotPresent | Never
```

```
imagePullSecrets:
- name: regcred # for private registries
kubectl create secret docker-registry regcred \
--docker-server=registry.example.com --docker-username=me --docker-
password=pw --docker-email=me@x.io
```

Pull policy rules: tag latest (or no tag) ⇒ Always; any other tag ⇒ IfNotPresent. On the exam, “use the local image without pulling” ⇒ imagePullPolicy: Never.

In podkit

- command-pod/Dockerfile is a complete, production-style recipe: slim base, one RUN per concern, tools installed, a non-root USER, WORKDIR, CMD ["sleep", "infinity"]. Build it with podkit image build, push with podkit image push.
- examples/java-mtls/Dockerfile shows the “one image, two programs” trick: the image’s CMD starts the server, and the client PodConfig overrides it with command: ["java", "/app/Client.java"].
- Every PodConfig has image.repository, image.tag and image.pullPolicy; the generator turns them into the container’s image: and imagePullPolicy:.

Exam tips

- Build in the directory the task names (cd /root/app && docker build -t ...). Check docker images before you move on.
- docker save -o FILE IMAGE:TAG — the flag is -o (or >), and the tag must match exactly.
- A wrong tag in a Pod shows as ErrImagePull/ImagePullBackOff; fix with kubectl set image pod/web web=nginx:1.27 or kubectl edit.
- Docs: the Dockerfile reference is **not** in the allowed list; know the eight instructions above.

Exercises (15 minutes)

1. Write a Dockerfile for a shell script hello.sh (echo hello from \$(hostname)) based on busybox:1.36, build it as hello:1.0, run it once, and save it to /tmp/hello.tar.
 2. Change the image so the script prints bye instead, build it as hello:1.1, and list both tags.
 3. Create a Pod that runs hello:1.1 from the local Docker daemon on a kind cluster without pulling (hint: podkit kind load + a pull policy).
-

Chapter 4 — Multi-container Pods

Plain words

Sometimes two programs must live *together*: they share the same network address (localhost) and can share a disk. Kubernetes has three classic patterns, and one newer feature:

Pattern	The helper container...	Example
Sidecar	runs next to the app, adds a feature	ship logs, serve TLS, cache
Ambassador	is a local proxy the app talks to on localhost	app → localhost:5432 → real database
Adapter	reshapes the app's output for someone else	convert logs to JSON for the collector
Init container	runs before the app, then exits	wait for a database, download config, fix permissions

Native **sidecar containers** (an init container with `restartPolicy: Always`) start before the app, keep running for the Pod's whole life and stop after the app — that is how Istio's Envoy proxy runs in the podkit SSO example.

Step by step: init container + shared volume

```
apiVersion: v1
kind: Pod
metadata:
  name: web-with-init
spec:
  volumes:
    - name: html
      emptyDir: {}                                # scratch disk shared by the
  containers, gone with the Pod
  initContainers:
    - name: fetch-page                             # runs first; the Pod waits until it
      image: busybox:1.36                          exits 0
      command: ["sh", "-c", "echo '<h1>hello from init</h1>' >
/work/index.html"]
      volumeMounts:
        - name: html
          mountPath: /work
  containers:
    - name: web
      image: nginx:1.27
      ports: [{ containerPort: 80 }]
      volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
kubectl apply -f web-with-init.yaml
kubectl get pod web-with-init                     # READY 0/1 + STATUS Init:0/1 while the
```

init runs, then Running

```
kubectl logs web-with-init -c fetch-page
kubectl exec web-with-init -c web -- cat /usr/share/nginx/html/index.html
```

Init containers run **in order**, one at a time, each must succeed. Common exam ask: “add an init container that waits until Service db resolves”:

```
initContainers:
- name: wait-for-db
  image: busybox:1.36
  command: ["sh", "-c", "until nslookup db.$(cat
/var/run/secrets/kubernetes.io/serviceaccount/namespace).svc.cluster.local; do
echo waiting; sleep 2; done"]
```

Step by step: sidecar that ships logs

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-logger
spec:
  volumes:
  - name: logs
    emptyDir: {}
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "i=0; while true; do echo \"$(date) line $i\" >>
/var/log/app.log; i=$((i+1)); sleep 2; done"]
    volumeMounts: [{ name: logs, mountPath: /var/log }]
  - name: log-shipper
    # the sidecar: reads what the app
writes
    image: busybox:1.36
    command: ["sh", "-c", "tail -F /var/log/app.log"]
    volumeMounts: [{ name: logs, mountPath: /var/log }]
  kubectl logs app-with-logger -c log-shipper -f # the sidecar's output = the
app's file
  kubectl logs app-with-logger --all-containers
```

The native-sidecar form (start first, live for the whole Pod, guaranteed ordering):

```
initContainers:
- name: log-shipper
  image: busybox:1.36
  restartPolicy: Always # <- this single line makes it a sidecar,
not a normal init
  command: ["sh", "-c", "tail -F /var/log/app.log"]
  volumeMounts: [{ name: logs, mountPath: /var/log }]
containers:
- name: app
...
```

Ambassador and adapter in one picture

```
app container —localhost:6379→ ambassador (proxy) → remote redis:6379
app container —writes /var/log→ adapter (reformat) → collector
```

Both are just “a second container in the same Pod” — what makes them a pattern is *who talks to whom*. When a task asks for an ambassador, the app’s config points at localhost.

Working with multi-container Pods

```
kubectl logs POD -c CONTAINER           # -c is required when there is more than
one container
kubectl exec -it POD -c CONTAINER -- sh
kubectl get pod POD -o jsonpath='{.spec.containers[*].name}'
kubectl describe pod POD                 # each container has its own section and
state
```

In podkit

- The Istio SSO example injects an Envoy **sidecar** into every Pod of the web and sso namespaces (metadata.namespaceLabels: {istio-injection: enabled}) — the sidecar terminates mutual TLS and enforces the JWT policy without changing the app.
- The command pod mounts an emptyDir at /work — the same “scratch disk shared by containers” idea; podkit command-pod sync fills it, playbooks read it.
- podgen puts small config files into a ConfigMap and mounts each with subPath (Chapter 5). A subPath mount is how you place one file into a directory that already has other files — the same trick you need when an init container must add a file next to nginx’s defaults.

Exam tips

- When editing a Pod that already exists you cannot add containers with `kubectl edit` (containers are immutable): `kubectl get pod x -o yaml > x.yaml`, edit, `kubectl replace --force -f x.yaml`.
- Shared data $\Rightarrow$ emptyDir + the **same volume name** in both containers’ volumeMounts.
- `readOnly: true` on the mount of the reading container is a nice touch graders cannot punish.
- Docs: *Workloads $\rightarrow$ Pods $\rightarrow$ Init Containers and Sidecar Containers*.

Exercises (25 minutes)

1. Create Pod two with containers main (nginx:1.27) and helper (busybox:1.36 running sleep 3600). Then print only the names of its containers.
2. Add to a new Pod fixer an init container that creates the file /data/ready in an emptyDir volume; the main container (busybox:1.36) must run `cat /data/ready && sleep 3600`. Prove the file exists from inside the main container.
3. Turn exercise 2’s helper into a **native sidecar** that runs `tail -F /data/app.log` and starts before the main container.
4. A Pod broken-init is stuck in Init:CrashLoopBackOff (create one whose init runs exit 1). Find the init container’s log with one command.

Chapter 5 — Configuration: ConfigMaps, Secrets, volumes, resources

Plain words

Good containers are like good recipes: the same image works in dev and prod because the *settings* come from outside. Kubernetes gives you three ways to hand settings to a Pod:

Mechanism	For	Shows up in the container as
env	a handful of values	environment variables
ConfigMap	plain settings (up to 1 MiB)	env vars or files in a volume
Secret	passwords, tokens, keys, certificates	env vars or files (base64 in the API; not encryption)

And two kinds of disk:

Volume	Lives	Use for
emptyDir	as long as the Pod	scratch space, sharing between containers
PersistentVolumeClaim (PVC)	beyond the Pod	data that must survive restarts (databases, uploads)

Finally, **requests and limits** tell the scheduler how much CPU and memory a container needs and may use; **LimitRange** and **ResourceQuota** are the namespace's house rules.

Step by step: ConfigMaps

```
kubectl create configmap app-cfg --from-literal=MODE=prod --from-literal=LOG_LEVEL=info
kubectl create configmap app-files --from-file=config.ini           # key =
file name
kubectl create configmap app-files2 --from-file=settings=config.ini # key =
settings
kubectl create configmap app-env --from-env-file=app.env           #
KEY=VALUE lines
kubectl get cm app-cfg -o yaml; kubectl describe cm app-cfg
```

Three ways to consume a ConfigMap:

```
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "env | grep -E 'MODE|LOG'; cat /etc/app/config.ini;
sleep 3600"]
      env:
        - name: MODE                                     # 1. one key -> one variable
          valueFrom:
            configMapKeyRef:
              name: app-cfg
              key: MODE
```

```

    envFrom:
      - configMapRef:
          name: app-cfg
          prefix: CFG_
        # 2. every key -> a variable
        # optional: CFG_MODE,
        CFG_LOG_LEVEL
      volumeMounts:
        - name: cfg
          mountPath: /etc/app
          # 3. every key -> a file
          readOnly: true
    volumes:
      - name: cfg
        configMap:
          name: app-files
          items:
            # optional: pick keys and rename
            - key: config.ini
              path: config.ini
files

```

Files in a volume are **updated live** when the ConfigMap changes (may take a minute); environment variables are **not** — the Pod must restart. Mounting with subPath also freezes the file (no live update), but lets you add one file to a directory without hiding what is already there:

```

volumeMounts:
  - name: cfg
    mountPath: /etc/nginx/conf.d/app.conf
    subPath: app.conf

```

Step by step: Secrets

```

kubectl create secret generic db --from-literal=user=app --from-
literal=password='S3cr3t!'
kubectl create secret generic tls-files --from-file=tls.crt --from-file=tls.key
kubectl create secret tls web-tls --cert=tls.crt --key=tls.key # type
kubernetes.io/tls
kubectl create secret docker-registry regcred --docker-server=... --docker-
username=... --docker-password=...
kubectl get secret db -o jsonpath='{.data.password}' | base64 -d; echo
kubectl get secret db -o go-template='{{index .data "password" |
base64decode}}'

```

Secrets are consumed exactly like ConfigMaps — swap configMapKeyRef → secretKeyRef, configMapRef → secretRef, configMap: → secret: (with secretName:):

```

env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef: { name: db, key: password }
envFrom:
  - secretRef: { name: db }
volumes:
  - name: creds
    secret:
      secretName: db
      defaultMode: 0400

```

Writing a Secret by hand: use `stringData`: (plain text; Kubernetes base64-encodes it for you) instead of `data`: (must be base64):

```
apiVersion: v1
kind: Secret
metadata: { name: db }
type: Opaque
stringData:
  password: S3cr3t!
```

Remember: base64 is an encoding, not encryption. Anyone who can get `secret` can read it — that is what RBAC (Chapter 9) is for.

Step by step: volumes that survive

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]      # RWO one node | ROX many read | RWX many
                                     # read-write | RWOP one pod
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard          # omit to use the default class; "" = only
                                     # pre-created PVs
                                     # in the Pod
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data
containers:
  - name: app
    volumeMounts:
      - name: data
        mountPath: /var/lib/app
kubectl get pvc,pv,storageclass
kubectl describe pvc data             # Pending? no StorageClass, or no PV matches
                                     # size/mode
```

A **PersistentVolume** is the actual disk; a claim asks for one. With a `StorageClass` the PV is created for you (dynamic provisioning). Without one, an admin creates PVs by hand (the exam sometimes gives you a PV and asks for a matching PVC — match `accessModes`, `size ≤ PV`, and `storageClassName`).

Other volumes you should recognise:

```
volumes:
  - name: scratch
    emptyDir:
      medium: Memory                  # RAM-backed
      sizeLimit: 64Mi
  - name: host
    hostPath:
      path: /var/log                  # a directory on the node (tests/demos only)
      type: Directory
  - name: cache
```

```
ephemeral:                # a PVC that lives and dies with the Pod
  volumeClaimTemplate:
    spec:
      accessModes: [ReadWriteOnce]
      resources: { requests: { storage: 1Gi } }
```

Step by step: requests, limits, quotas

```
resources:
  requests: { cpu: 250m, memory: 128Mi } # what the scheduler reserves
  limits:   { cpu: 500m, memory: 256Mi } # what the container may burst to
```

- 250m = a quarter of a CPU core. 128Mi = 128 mebibytes (128M would be megabytes — use Mi).
- Over the **memory** limit ⇒ the container is killed (OOMKilled) and restarted. Over the **CPU** limit ⇒ throttled, not killed.
- No requests at all ⇒ “BestEffort” QoS, first to be evicted. requests = limits ⇒ “Guaranteed”.

```
kubectl set resources deployment web -c nginx --requests=cpu=100m,memory=64Mi
--limits=cpu=200m,memory=128Mi
kubectl top pod; kubectl top node          # needs metrics-server
```

Namespace rules:

```
apiVersion: v1
kind: LimitRange                # defaults and bounds per container
metadata: { name: defaults, namespace: dev }
spec:
  limits:
    - type: Container
      default:      { cpu: 500m, memory: 256Mi } # limit if the Pod does
not say
      defaultRequest: { cpu: 100m, memory: 128Mi } # request if the Pod does
not say
      max:           { cpu: "2", memory: 1Gi }
  ---
apiVersion: v1
kind: ResourceQuota            # total for the whole namespace
metadata: { name: team-quota, namespace: dev }
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    pods: "20"
kubectl create quota team-quota --hard=pods=20,requests.cpu=4 -n dev
kubectl describe quota -n dev    # Used vs Hard
```

Trap: when a ResourceQuota sets requests.*, every Pod in the namespace **must** declare requests or it is rejected (“failed quota: must specify requests.cpu”). A LimitRange with defaults fixes that.

In podkit

- podkit secret sync BUILD_DIR reads values from the cloud secret store and writes Secret <name>-env; the Deployment consumes it with envFrom: [{secretRef: ...}] — see podgen/k8s.py — deployment().

- PodConfig files: become ConfigMap <name>-files, mounted read-only with subPath (one file per key, key = path with / - __).
- A checksum of the files/secrets is written to a Pod-template annotation (podkit.dev/files-checksum), so changing a ConfigMap rolls the Pods — the standard trick for “env vars don’t update live”.
- resources.requests/limits are always set (schema defaults 100m/128Mi → 500m/256Mi). terraform/modules/pod/variables.tf documents why: predictable scheduling and bills.
- The command pod’s /work is an emptyDir; podkit tls secrets hold keystore.jks (binary data) — a Secret can carry binary files, not just strings.

Exam tips

- --from-literal for values, --from-file for files, --from-env-file for a KEY=VALUE file.
- “Expose all keys as env” ⇒ envFrom. “Mount as files under /x” ⇒ volume. “Only key K as file /x/K” ⇒ items or subPath.
- Pods do not restart when a Secret used as env changes: `kubectl rollout restart deployment X`.
- `kubectl explain pod.spec.volumes.persistentVolumeClaim` if you forget claimName.
- Docs: *Configure Pods* → *ConfigMaps, Secrets, Persistent Volumes, Manage Resources*.

Exercises (30 minutes)

1. Create ConfigMap web-cfg with COLOR=blue and SIZE=large, and Secret web-sec with token=abc123. Run Pod web (busybox:1.36, sleep 3600) that exposes **all** ConfigMap keys as env vars and the token as API_TOKEN. Prove it with `kubectl exec web -- env`.
 2. Mount the same ConfigMap as files under /etc/web in a second Pod web-files; show the content of /etc/web/COLOR.
 3. A PersistentVolume pv-data (1Gi, ReadWriteOnce, storageClassName: manual, hostPath /tmp/data) exists — create it. Write a PVC data-claim that binds to it and a Pod writer that mounts it at /data and writes hello into /data/hi. Check the PVC is Bound.
 4. Namespace limited has a ResourceQuota pods=2, requests.cpu=500m. Create it, then create a Deployment q with 3 replicas of nginx:1.27 requesting 200m CPU. How many Pods run, and why? (Look at `kubectl describe deploy q` and `kubectl describe quota -n limited`.)
 5. Create a LimitRange in limited that gives containers a default request of 100m/64Mi and a default limit of 200m/128Mi. Create a Pod without resources and check what it got.
-

Day 1 solutions

Chapter 1

```
# 1
kubectl create namespace ckad-1
kubectl run hello --image=busybox:1.36 -n ckad-1 -- sleep 3600
kubectl get pod hello -n ckad-1 -o jsonpath='{.status.podIP}'
# 2
kubectl create deployment api --image=nginx:1.27 --replicas=2 $do > api.yaml
# add tier: backend under spec.template.metadata.labels, then:
kubectl apply -f api.yaml; kubectl get pods -l tier=backend
# 3
kubectl api-resources --namespaced=false
# 4
kubectl explain pod.spec.dnsPolicy
```

Chapter 2

```
# 1
kubectl run sleeper --image=busybox:1.36 --restart=OnFailure -- sleep 300
# 2
kubectl create namespace shop
kubectl create deployment store --image=nginx:1.26 --replicas=4 -n shop
kubectl scale deployment store --replicas=2 -n shop
# 3
kubectl create job countdown --image=busybox:1.36 $do -- sh -c 'for i in 5 4 3
2 1; do echo $i; done' > job.yaml
# add under spec: completions: 3 parallelism: 3 ttlSecondsAfterFinished:
30
kubectl apply -f job.yaml; kubectl logs -l job-name=countdown --tail=5
# 4
kubectl create cronjob ping --image=busybox:1.36 --schedule="*/10 * * * *" $do
-- date > cj.yaml
# add under spec: concurrencyPolicy: Forbid successfulJobsHistoryLimit: 1
kubectl apply -f cj.yaml; kubectl create job ping-now --from=cronjob/ping
# 5
kubectl get deployment store -n shop -o yaml > ds.yaml
# change kind: DaemonSet, name: store-ds, delete
replicas/strategy/status/resourceVersion/uid
kubectl apply -f ds.yaml
```

Chapter 3

```
FROM busybox:1.36
COPY hello.sh /hello.sh
CMD ["sh", "/hello.sh"]
printf 'echo hello from $(hostname)\n' > hello.sh
docker build -t hello:1.0 . && docker run --rm hello:1.0 && docker save -o
/tmp/hello.tar hello:1.0
sed -i 's/hello/bye/' hello.sh && docker build -t hello:1.1 . && docker images
hello
podkit kind load hello:1.1
kubectl run hello --image=hello:1.1 --image-pull-policy=Never --restart=Never
```

Chapter 4

```
# 1
kubectl run two --image=nginx:1.27 $do > two.yaml # add a second container
"helper" (busybox, sleep 3600); rename first to main
kubectl apply -f two.yaml; kubectl get pod two -o
jsonpath='{.spec.containers[*].name}'
# 2 – Pod with emptyDir "data", initContainer: sh -c 'touch /data/ready',
container: sh -c 'cat /data/ready && sleep 3600'
kubectl exec fixer -- ls /data/ready
# 3 – move the helper to initContainers with restartPolicy: Always
# 4
kubectl logs broken-init -c <init-container-name>
```

Chapter 5

```
# 1
kubectl create configmap web-cfg --from-literal=COLOR=blue --from-
literal=SIZE=large
kubectl create secret generic web-sec --from-literal=token=abc123
kubectl run web --image=busybox:1.36 $do -- sleep 3600 > web.yaml
# add: envFrom: [{configMapRef: {name: web-cfg}}]
# env: [{name: API_TOKEN, valueFrom: {secretKeyRef: {name: web-sec, key:
token}}}]
kubectl apply -f web.yaml; kubectl exec web -- env | grep -E 'COLOR|SIZE|
API_TOKEN'
# 2 – volumes: [{name: cfg, configMap: {name: web-cfg}}]; volumeMounts: [{name:
cfg, mountPath: /etc/web}]
kubectl exec web-files -- cat /etc/web/COLOR
# 3 – PV (hostPath) + PVC with storageClassName: manual, accessModes
[ReadWriteOnce], 1Gi; Pod mounts claimName data-claim
kubectl get pvc data-claim # STATUS Bound
# 4 – only 2 Pods run: the quota allows pods=2 and requests.cpu=500m; describe
deploy shows "exceeded quota"
# 5 – LimitRange type Container with default/defaultRequest; kubectl describe
pod X shows Requests 100m/64Mi, Limits 200m/128Mi
```

Day 2 — Deploy, observe, secure

Chapter 6 — Deployments: rolling updates, rollbacks, blue/green, canary

Plain words

A **Deployment** keeps N Pods running and knows how to change them safely. Change the image and it makes a new ReplicaSet, starts new Pods a few at a time, and retires old ones — a **rolling update**. Every change is a numbered **revision**, and you can roll back to any of them.

Two other strategies are done *with* Deployments and Services, not by a special object:

- **Blue/green** — run the new version (green) next to the old one (blue), test it, then flip the Service's selector so all traffic jumps at once. Instant rollback: flip it back.
- **Canary** — run a *few* Pods of the new version behind the same Service as the old ones, so a small share of users hits it. Happy? Scale the canary up and the old one down.

Step by step: rolling update and rollback

```
kubectl create deployment web --image=nginx:1.26 --replicas=4
kubectl set image deployment/web nginx=nginx:1.27          # container name =
image name without tag/registry
kubectl rollout status deployment/web                      # waits until done
kubectl rollout history deployment/web                     # revisions 1, 2
kubectl rollout history deployment/web --revision=2
kubectl rollout undo deployment/web                       # back to the
previous revision
kubectl rollout undo deployment/web --to-revision=1
kubectl rollout pause deployment/web; kubectl rollout resume deployment/web
kubectl rollout restart deployment/web                    # new Pods, same
spec (reload ConfigMaps/Secrets)
kubectl annotate deployment/web kubernetes.io/change-cause="upgrade to 1.27"
# shows in history
```

The strategy lives in `spec.strategy`:

```
spec:
  replicas: 4
  revisionHistoryLimit: 5      # how many old ReplicaSets to keep (for undo)
  minReadySeconds: 5          # a new Pod counts as ready only after 5 s of
being Ready
  progressDeadlineSeconds: 600 # mark the rollout failed after 10 min
  strategy:
    type: RollingUpdate        # or Recreate: kill all, then start all
(downtime, but no mixed versions)
    rollingUpdate:
      maxSurge: 1              # how many extra Pods may exist during the
update (number or %)
      maxUnavailable: 0        # how many Pods may be missing (0 = zero
downtime)
```

- `maxSurge`: 25%, `maxUnavailable`: 25% are the defaults.
- `maxUnavailable`: `0 + maxSurge: 1` = safest: always all replicas serving.
- `Recreate` is for apps that cannot run two versions at once (schema migrations, single-writer).

Watch a rollout to understand it:

```
kubectl set image deployment/web nginx=nginx:1.28 && kubectl get rs -w
```

A failed rollout (bad image) stays half-done — `rollout status` reports `ProgressDeadlineExceeded`, the old Pods keep serving; `rollout undo` fixes it.

Step by step: scaling and autoscaling

```
kubectl scale deployment web --replicas=6
kubectl autoscale deployment web --min=2 --max=10 --cpu-percent=70 #
HorizontalPodAutoscaler
kubectl get hpa
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: web }
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

The HPA needs **CPU requests** on the containers (utilisation is measured against requests) and `metrics-server` in the cluster.

Step by step: blue/green

```
kubectl create deployment web-blue --image=nginx:1.26 --replicas=3
kubectl create deployment web-green --image=nginx:1.27 --replicas=3
kubectl label deployment web-blue --overwrite version=blue # (labels on the
Pod template matter, see below)
```

Give the Pod templates a version label and point the Service at one version:

```
# web-blue: spec.template.metadata.labels: { app: web, version: blue }
# web-green: spec.template.metadata.labels: { app: web, version: green }
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web, version: blue } # all traffic to blue
  ports: [{ port: 80 }]
```

Test green privately (`kubectl port-forward deploy/web-green 8080:80`), then switch:

```
kubectl patch service web -p '{"spec":{"selector":
{"app":"web","version":"green"}}}'
```

Roll back = patch it back to blue. Cost: double the Pods during the switch.

Step by step: canary

Same app: `web` label on **both** Deployments; the Service selects only app: `web`, so traffic is split by Pod count:

```
kubectl create deployment web-stable --image=nginx:1.26 --replicas=9 #
template labels: app=web, track=stable
kubectl create deployment web-canary --image=nginx:1.27 --replicas=1 #
template labels: app=web, track=canary
kubectl expose deployment web-stable --name=web --port=80 --selector=app=web
```

1 of 10 Pods is the canary $\Rightarrow$ roughly 10 % of requests. Promote by scaling: `kubectl scale deployment web-canary --replicas=5`; `kubectl scale deployment web-stable --replicas=5`, finally set the stable Deployment's image to the new version and delete the canary. Precise traffic percentages (5 % with 3 Pods) need a service mesh — that is what the Istio VirtualService in the podkit SSO example can do with `weight`:

In podkit

- `podgen/k8s.py` sets `RollingUpdate` with `maxSurge: 1`, `maxUnavailable: 0` and `revisionHistoryLimit: 2` — zero-downtime updates without keeping ten old ReplicaSets.
- The Pod template carries checksum annotations for files and secrets; changing a value changes the annotation, and that change alone triggers a rolling update. Equivalent to `kubectl rollout restart`, but automatic.
- `podkit pod restart BUILD_DIR` is `kubectl rollout restart + rollout status`; `podkit pod scale BUILD_DIR N` is `kubectl scale`.
- `autoscaling.enabled: true` in a PodConfig emits an `autoscaling/v2` HPA and **omits** replicas from the Deployment so the HPA owns the count — a subtle point worth remembering.
- The Terraform pod module does the same (`replicas = var.autoscaling.enabled ? null : ...`).

Exam tips

- `kubectl set image deployment/NAME CONTAINER=IMAGE:TAG` — you need the **container name** (`kubectl get deploy NAME -o jsonpath='{.spec.template.spec.containers[*].name}'`).
- “Roll back to the previous version” $\Rightarrow$ `rollout undo`. “To revision 2” $\Rightarrow$ `--to-revision=2`.
- “No more than one Pod unavailable and no extra Pods” $\Rightarrow$ `maxUnavailable: 1`, `maxSurge: 0`.
- A task that says “update the image and record the change” $\Rightarrow$ set the image and add the `kubernetes.io/change-cause` annotation (the old `--record` flag is gone).
- Docs: *Workloads – Deployments* (has every strategy field), *Horizontal Pod Autoscaling*.

Exercises (25 minutes)

1. Create Deployment `shop` with `nginx:1.25` and 3 replicas. Update it to `nginx:1.26`, then to `nginx:1.27`, annotating each change cause. Show the history, then roll back to the 1.26 revision.
2. Change `shop` so that updates never remove a Pod before its replacement is ready and never run more than 4 Pods. Verify with `kubectl get deploy shop -o jsonpath='{.spec.strategy}'`.

3. Set the image of shop to `nginx:doesnotexist`. What does `rollout status` say after a minute? How many Pods are still serving the old version? Fix it with one command.
 4. Build a blue/green setup for `nginx:1.26` (blue) and `nginx:1.27` (green) behind Service front in namespace `bg`, then switch traffic to green with a `kubectl patch`.
 5. Create an HPA for shop with 2–6 replicas at 60 % CPU (add CPU requests first if missing).
-

Chapter 7 — Helm and Kustomize

Plain words

Writing every YAML file by hand does not scale. Two tools package and customise manifests:

- **Helm** is the app store. A **chart** is a folder of templates plus a `values.yaml`; a **release** is a chart installed into a namespace with your values. You mostly *use* charts other people wrote (Istio, Keycloak, nginx-ingress). Exam tasks: add a repo, install a chart with custom values, upgrade, roll back, uninstall.
- **Kustomize** is built into `kubectl`. You keep plain YAML in a base folder and small **overlays** (dev, prod) that patch it: change replicas, add a name prefix, swap an image, set labels. Exam tasks: build/apply an overlay, add a patch.

Step by step: Helm

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm repo update
helm search repo nginx                # charts in your repos
helm search repo bitnami/nginx --versions  # all chart versions
helm search hub keycloak              # Artifact Hub (needs internet)
helm show values bitnami/nginx > values.yaml # every knob with its default
helm show chart bitnami/nginx

helm install web bitnami/nginx -n apps --create-namespace \
  --set replicaCount=2 --set service.type=ClusterIP # release "web"
helm install web bitnami/nginx -n apps -f values.yaml --version 18.2.0
helm list -A                                # releases, revisions, status
helm status web -n apps
helm get values web -n apps                 # the values you supplied
helm get manifest web -n apps               # the rendered YAML that was
applied

helm upgrade web bitnami/nginx -n apps --set replicaCount=3 # revision
2
helm upgrade --install web bitnami/nginx -n apps -f values.yaml # install
or upgrade (idempotent)
helm history web -n apps
helm rollback web 1 -n apps                 # back to revision 1 (creates
revision 3)
helm uninstall web -n apps

helm template web bitnami/nginx -f values.yaml > rendered.yaml # render
without installing
helm install web bitnami/nginx --dry-run --debug                # test against
the cluster
helm pull bitnami/nginx --untar                                # download the chart to read it
```

Value precedence: `--set` beats `-f values.yaml` beats the chart's defaults. Nested keys use dots (`--set service.port=8080`), lists use brackets (`--set ingress.hosts[0]=x.io`).

A chart on disk looks like:

```
mychart/
Chart.yaml          # name, version, appVersion
```

```

values.yaml          # defaults
templates/          # YAML with {{ .Values.replicaCount }} placeholders
  deployment.yaml
  service.yaml
  _helpers.tpl

```

helm create mychart scaffolds one; helm lint mychart checks it; helm install x ./mychart installs from the folder.

Step by step: Kustomize

```

app/
  base/
    kustomization.yaml
    deployment.yaml
    service.yaml
  overlays/
    dev/
      kustomization.yaml
    prod/
      kustomization.yaml
      replicas.yaml
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
labels:                                     # (commonLabels is the older, deprecated name)
  - pairs: { app: web }
    includeSelectors: true
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namespace: prod
namePrefix: prod-
images:
  - name: nginx                             # image name as written in the base
    newTag: "1.27"
replicas:
  - name: web
    count: 5
patches:
  - path: replicas.yaml                     # a strategic-merge patch: partial YAML with
the same name/kind
  - target: { kind: Deployment, name: web }
    patch: |-                               # an inline JSON6902 patch
      - op: add
        path: /spec/template/spec/containers/0/env
        value: [{ name: MODE, value: prod }]
configMapGenerator:
  - name: web-cfg
    literals: [COLOR=blue]                 # creates web-cfg-<hash>; references are
rewritten automatically
secretGenerator:

```

```

- name: web-sec
  literals: [token=abc]
kubectl kustomize overlays/prod           # render only
kubectl apply -k overlays/prod            # build + apply
kubectl delete -k overlays/prod
kubectl diff -k overlays/prod             # what would change

```

In podkit

- podkit istio install is three helm upgrade --install commands (istio/base, istio/istiod, istio/gateway) with rendered values files — see scripts/resources/istio.sh. Read it once; it is the shape of every Helm exam task: repo add, update, upgrade --install with -n, --create-namespace, -f values.yaml, --wait.
- podkit istio destroy runs helm uninstall in reverse order.
- podkit itself is a generator like Kustomize: one source of truth (PodConfig), rendered per environment (--provider aws|azure|gcp|local). The build directory is the “overlay”.

Exam tips

- Always pass -n NAMESPACE to Helm; releases are namespaced, and helm list without -A hides releases in other namespaces.
- helm upgrade --install is safe to repeat; plain helm install fails if the release exists.
- Read the task for the exact chart **version** (--version) and value names; use helm show values when unsure of a key.
- Kustomize: the overlay’s resources: points at the base folder; kubectl apply -k takes a **directory**, not a file.
- Docs: helm.sh/docs is allowed; kubectl’s Kustomize page is under *Tasks — Manage Kubernetes Objects — Declarative Management using Kustomize*.

Exercises (25 minutes)

1. Add the bitnami repo, install chart bitnami/nginx as release site into namespace helm-1 with 2 replicas and service type ClusterIP. List releases in all namespaces.
2. Upgrade site to 3 replicas, then roll back to revision 1. Show the history.
3. Render the chart to a file without installing, and find the Deployment’s image line.
4. Create a Kustomize base with a Deployment hello (nginx:1.26, 1 replica) and a prod overlay that sets namespace prod, prefix prod-, 3 replicas and image tag 1.27. Apply it and verify the Deployment is named prod-hello with the new tag.
5. Uninstall site and delete the kustomize overlay’s objects.

Chapter 8 — Observability and maintenance: probes, logs, debugging, deprecations

Plain words

Kubernetes cannot know whether your app is *healthy* unless you tell it how to check. **Probes** are those checks. **Logs** and **events** are how the cluster talks back to you. **Debugging** is reading them in the right order. And because the API evolves, old apiVersions get **deprecated** — a task may hand you an old manifest and ask you to fix it.

Step by step: probes

Probe	Question	On failure
startupProbe	“has the app finished starting?”	keep waiting; the other probes are paused
readinessProbe	“can it take traffic right now?”	Pod removed from Service endpoints (READY 0/1), not restarted
livenessProbe	“is it still alive?”	container is restarted

Three ways to ask, usable in any probe:

```
containers:
- name: web
  image: nginx:1.27
  ports: [{ containerPort: 80, name: http }]
  startupProbe:
    httpGet: { path: /, port: http }
    failureThreshold: 30           # 30 × 5 s = up to 150 s to start
    periodSeconds: 5
  readinessProbe:
    httpGet: { path: /healthz, port: 80, httpHeaders: [{ name: X-Probe,
value: k8s }] }
    initialDelaySeconds: 3
    periodSeconds: 5
    timeoutSeconds: 1
    successThreshold: 1
    failureThreshold: 3
  livenessProbe:
    tcpSocket: { port: 80 }        # can I open a TCP connection?
    initialDelaySeconds: 10
    periodSeconds: 10
  livenessProbe:
    exec:                          # run a command; exit 0 = healthy
    command: ["sh", "-c", "test -f /tmp/healthy"]
  livenessProbe:
    grpc:                          # gRPC health checking protocol
    port: 9090
```

Timing rules: the probe first runs after `initialDelaySeconds`, then every `periodSeconds`; it fails after `failureThreshold` consecutive failures; each try may take `timeoutSeconds`.

```
kubectl describe pod web | grep -A3 -E 'Liveness|Readiness|Startup'
kubectl get events --field-selector involvedObject.name=web # "Liveness
probe failed: ..."
```

A classic exam task: “the Pod restarts every 20 seconds” ⇒ the liveness probe is wrong (wrong port/path, or the app needs longer than `initialDelaySeconds`). Another: “the Service has no endpoints although the Pod runs” ⇒ readiness probe failing.

Step by step: logs and metrics

```
kubectl logs web # current container
kubectl logs web -c helper # named container
kubectl logs web --previous # the crashed one (CrashLoopBackOff!)
kubectl logs web -f --tail=50 # follow
kubectl logs web --since=5m
kubectl logs -l app=web --all-containers --prefix # every Pod with the label
kubectl logs deployment/web # one Pod of the Deployment
kubectl top pod -n apps --sort-by=memory
kubectl top pod web --containers
kubectl top node
```

Logs are whatever the process writes to **stdout/stderr**. If an app writes to a file, you need a sidecar to tail it (Chapter 4) — the exam likes that combination.

Step by step: the debugging order

1. `kubectl get pods` — status and READY.
2. `kubectl describe pod X` — scroll to **Events**: scheduling errors, image pull errors, probe failures, OOMKilled, volume/ConfigMap missing.
3. `kubectl logs X [-c C] [--previous]` — the app’s own words.
4. `kubectl get events -n NS --sort-by=.lastTimestamp` — the namespace’s timeline.
5. `kubectl exec -it X -- sh` — look around (`env`, `cat`, `wget -q0- localhost:8080`).
6. No shell in the image? **Ephemeral container**:

```
kubectl debug -it web --image=busybox:1.36 --target=web # shares the
process namespace of container "web"
kubectl debug web --copy-to=web-debug --container=web -- sh # a copy of the
Pod running sh instead
kubectl debug node/kind-podkit-control-plane -it --image=busybox:1.36 # a Pod
on the node with the host filesystem at /host
```

Symptoms and causes you should recognise instantly:

Symptom	Likely cause
Pending, event “0/3 nodes are available: insufficient cpu”	requests too big; lower them or add nodes
Pending, “persistentvolumeclaim not found” / PVC Pending	claim or StorageClass missing
CreateContainerConfigError	ConfigMap/Secret key referenced in env does not exist
ImagePullBackOff	typo in image/tag, or private registry without

Symptom	Likely cause
	imagePullSecrets
CrashLoopBackOff	app exits: wrong command, missing config, port in use — read logs --previous
OOMKilled (in describe → Last State)	memory limit too low
READY 0/1 forever	readiness probe fails (path/port)
Service has no endpoints	label mismatch between Service selector and Pods, or Pods not Ready
Forbidden on kubectl	RBAC; check with kubectl auth can-i

Useful one-liners:

```
kubectl get pods -A --field-selector status.phase!=Running
kubectl get pods -o custom-
columns='NAME:.metadata.name,RESTARTS:.status.containerStatuses[0].restartCount
,IMAGE:.spec.containers[0].image'
kubectl get pod web -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'
kubectl get endpoints web; kubectl get endpointslices -l kubernetes.io/service-
name=web
```

Step by step: API deprecations

Every kind has an `apiVersion`; over time old ones are removed. Examples you may still meet in old manifests: `extensions/v1beta1 Deployment` (now `apps/v1`), `extensions/v1beta1 Ingress` (now `networking.k8s.io/v1`), `batch/v1beta1 CronJob` (now `batch/v1`), `policy/v1beta1 PodDisruptionBudget` (now `policy/v1`), `autoscaling/v2beta2 HPA` (now `autoscaling/v2`).

```
kubectl api-resources | grep -i ingress          # which group/version the
cluster serves
kubectl api-versions
kubectl explain ingress --api-version=networking.k8s.io/v1
kubectl apply -f old.yaml                       # "no matches for kind Ingress in version
extensions/v1beta1" -> fix the apiVersion
kubectl convert -f old.yaml --output-version networking.k8s.io/v1 # if the
convert plugin is installed
```

The server prints a **Warning**: line when you use a deprecated but still served version — take it as an instruction. The *Deprecated API Migration Guide* on kubernetes.io lists what changed in each release (fields may have moved, e.g. `Ingress serviceName` → `service.name`).

In podkit

- Every generated Deployment has a liveness **and** readiness probe from `spec.probe` (`http` or `tcp`); the Java server example uses `tcp` because a TLS listener cannot answer a plain HTTP probe — a real-world reason to pick the probe type.

- `podkit pod status BUILD_DIR` prints the Deployment, Pods, Service and the last five events — the debugging order above in one command; `podkit pod logs BUILD_DIR -f` streams logs.
- `podkit doctor` is a checklist of “can I even talk to the cluster?” (`kubectl auth can-i` is used to test namespace permissions) — the same first step you take when a task’s `kubectl` fails.
- `podkit`’s own manifests use current `apiVersions` only (`apps/v1`, `batch/v1`, `networking.k8s.io/v1`, `autoscaling/v2`, `networking.istio.io/v1`).

Exam tips

- “Restart when unhealthy” ⇒ liveness. “Only receive traffic when ready” ⇒ readiness. “Slow to start” ⇒ startup probe (or a large `initialDelaySeconds`).
- `httpGet.port` can be a number or a **port name**; a wrong name is a silent failure.
- `kubectl logs --previous` is the single most useful flag for `CrashLoopBackOff`.
- You cannot edit probes on a running Pod: `dump`, `edit`, `replace --force`. On a Deployment you can (`kubectl edit deploy`) and it rolls out.
- Docs: *Configure Liveness, Readiness and Startup Probes, Debug Pods, Deprecated API Migration Guide*.

Exercises (25 minutes)

1. Create Pod probed (`nginx:1.27`) with a readiness probe on / port 80 every 5 s (initial delay 2 s) and a liveness probe that opens TCP port 80 every 10 s. Show the probe config from `describe`.
2. Create Pod flaky (`busybox:1.36`) running `touch /tmp/ok; sleep 30; rm /tmp/ok; sleep 3600` with an exec liveness probe `cat /tmp/ok` every 5 s. Watch the restart count grow; explain why.
3. Create Deployment crasher (`busybox:1.36`, command `sh -c 'echo boom; exit 1'`). Using only `kubectl`, capture the reason it fails into `/tmp/reason.txt`.
4. The manifest below is rejected. Fix it so it applies:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata: { name: old }
spec:
  template:
    metadata: { labels: { app: old } }
    spec: { containers: [{ name: web, image: nginx:1.27 }] }
```

5. Pod `noshell` runs `gcr.io/distroless/static:nonroot-style` image without a shell (use registry.k8s.io/pause:3.9). Start an ephemeral `busybox` container attached to it and run `ps`.

Chapter 9 — Security: ServiceAccounts, RBAC, SecurityContext, admission, CRDs

Plain words

Three questions guard every request to the API server:

1. **Authentication** — who are you? (a user with a certificate, or a Pod with a **ServiceAccount** token)
2. **Authorization** — are you allowed? (**RBAC**: Roles say *what*, RoleBindings say *who*)
3. **Admission** — is the object acceptable? (built-in checks and policies such as Pod Security Standards; also where defaults and quotas are enforced)

Inside the Pod, the **SecurityContext** decides *how* the process runs: which user, which Linux capabilities, whether it can write to its root filesystem or become root.

And **Custom Resource Definitions** (CRDs) let people add new kinds (Certificate, VirtualService, Kafka) that an **operator** turns into real things.

Step by step: ServiceAccounts

Every Pod runs as a ServiceAccount (default unless you say otherwise). The token is mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token`; that is how a Pod calls the API.

```
kubectl create serviceaccount app-sa
kubectl get sa
kubectl create token app-sa --duration=1h           # a short-lived token
(no Secret objects any more)
kubectl set serviceaccount deployment web app-sa    # switch a
Deployment's SA (rolls out)
spec:
  serviceAccountName: app-sa
  automountServiceAccountToken: false               # the app never calls the API? do
not mount the token
```

Step by step: RBAC

```
kubectl create role pod-reader --verb=get,list,watch --resource=pods -n dev
kubectl create rolebinding read-pods --role=pod-reader --
serviceaccount=dev:app-sa -n dev
kubectl create rolebinding read-pods-jane --role=pod-reader --user=jane -n dev
kubectl create clusterrole node-viewer --verb=get,list --resource=nodes
kubectl create clusterrolebinding view-nodes --clusterrole=node-viewer --
user=jane
kubectl create clusterrolebinding jane-view --clusterrole=view --user=jane
# built-in: view, edit, admin, cluster-admin
kubectl auth can-i list pods --as=system:serviceaccount:dev:app-sa -n dev
kubectl auth can-i create deployments --as=jane -n dev
kubectl auth can-i --list --as=jane -n dev
kubectl get role,rolebinding -n dev; kubectl describe role pod-reader -n dev
apiVersion: rbac.authorization.k8s.io/v1
kind: Role                                     # namespaced; ClusterRole for cluster-wide
or non-namespaced things
metadata: { name: pod-reader, namespace: dev }
```

```

rules:
  - apiGroups: [""]                # "" = core (pods, services, configmaps, secrets)
    resources: ["pods", "pods/log"] # subresources count separately
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "patch"]
    resourceNames: ["web"]        # optional: only this object
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: dev }
subjects:
  - kind: ServiceAccount
    name: app-sa
    namespace: dev
  - kind: User
    name: jane
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role                # a RoleBinding can also reference a ClusterRole (reusable rules, namespaced effect)
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

Verbs: get list watch create update patch delete deletecollection. * = all. Find the group of a resource with `kubectl api-resources` (the APIVERSION column: apps/v1 ⇒ group apps, v1 ⇒ core "").

Test from inside a Pod:

```

kubectl exec -it web -- sh -c 'TOKEN=$(cat
/var/run/secrets/kubernetes.io/serviceaccount/token); \
curl -sk -H "Authorization: Bearer $TOKEN"
https://kubernetes.default.svc/api/v1/namespaces/dev/pods | head'

```

Step by step: SecurityContext

```

spec:
  securityContext:                # Pod level: applies to all containers
    (and volumes for fsGroup)
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000                # files in volumes are group-owned by
2000
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: app
      image: busybox:1.36
      securityContext:            # container level: overrides the Pod
level
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true # app must write only to volumes (/tmp as
emptyDir)
        privileged: false
        capabilities:

```

```

        drop: ["ALL"]
        add: ["NET_BIND_SERVICE"]    # bind ports < 1024 without being root
kubectrl exec app -- id              # uid=1000 gid=3000
groups=2000
kubectrl exec app -- touch /x        # "Read-only file
system"
kubectrl get pod app -o jsonpath='{.spec.containers[0].securityContext}'

```

Common trap: `runAsNonRoot: true` with an image whose default user is root and no `runAsUser` ⇒ the container never starts (“container has `runAsNonRoot` and image will run as root”). Add `runAsUser: 1000` (any non-zero uid).

Step by step: Pod Security Standards (admission)

The built-in Pod Security Admission checks Pods against three levels — privileged, baseline, restricted — per namespace label:

```

kubectrl label namespace dev pod-security.kubernetes.io/enforce=restricted
kubectrl label namespace dev pod-security.kubernetes.io/warn=restricted

```

restricted requires roughly the container SecurityContext above (non-root, no privilege escalation, drop ALL capabilities, seccomp RuntimeDefault). A Pod that violates it is **rejected at creation** with a clear message telling you which fields to fix — read it, fix, apply.

Other admission concepts to recognise: **LimitRange/ResourceQuota** (Chapter 5) are enforced at admission; **ValidatingAdmissionPolicy** (CEL rules) and webhooks can reject or mutate objects; `kubectrl apply` shows their messages.

Step by step: CRDs and operators

```

kubectrl get crd                                # every custom kind installed
kubectrl get crd virtualservices.networking.istio.io -o yaml | head -40
kubectrl explain virtualservice.spec             # CRDs with a schema support
explain
kubectrl api-resources | grep istio
kubectrl get virtualservices -A                 # use the new kind like any
other

```

A CRD defines the shape; the **operator** (a Deployment watching those objects) does the work. Writing a CRD is rare on the exam; *using* an existing custom resource is common: “create a Foo object in namespace X with these fields” ⇒ `kubectrl explain foo.spec`, write the YAML, apply.

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: { name: backups.podkit.dev }          # <plural>.<group>
spec:
  group: podkit.dev
  scope: Namespaced
  names: { plural: backups, singular: backup, kind: Backup, shortNames: [bk] }
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object

```

```

    properties:
      spec:
        type: object
        properties:
          schedule: { type: string }
          keep: { type: integer }
apiVersion: podkit.dev/v1
kind: Backup
metadata: { name: nightly }
spec: { schedule: "0 2 * * *", keep: 7 }

```

In podkit

- Every generated pod has **its own ServiceAccount** with `automountServiceAccountToken: false`, and the cloud identity is bound to it by annotation (`podkit identity bind`) — least privilege in practice.
- The command pod gets a namespaced **Role + RoleBinding per target namespace** (`command-pod/k8s/rbac.yaml`, `podkit command-pod grant NS`): pods `get/list/watch`, pods `exec create`, secrets/configmaps `CRUD`, deployments `patch`. No `ClusterRole`, no wildcard — copy that shape for RBAC tasks.
- `podgen/k8s.py` writes the `SecurityContext` defaults from the Pod Security “restricted” profile: `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`, plus `runAsNonRoot/runAsUser/readOnlyRootFilesystem` from the `PodConfig`.
- `podkit namespace create NS --pss restricted` labels the namespace for Pod Security Admission.
- Istio installs dozens of CRDs (`Gateway`, `VirtualService`, `AuthorizationPolicy`, `PeerAuthentication`); manifests/`sso/app.yaml` uses three of them — real custom resources created by “an operator” (istiod).

Exam tips

- “Allow the ServiceAccount X to list Pods in namespace Y” $\Rightarrow$ **Role + RoleBinding in Y**, subject `serviceaccount=Y:X`. Verify with `auth can-i --as=system:serviceaccount:Y:X`.
- Cluster-wide resources (nodes, namespaces, PVs) need a **ClusterRole + ClusterRoleBinding**.
- Container-level `securityContext` wins over Pod-level. `runAsNonRoot` needs a numeric user.
- “Run as user 1000 and read-only root FS, but allow writing /tmp” $\Rightarrow$ add an `emptyDir` at /tmp.
- Docs: *RBAC*, *Configure a Security Context*, *Pod Security Standards*, *Custom Resources*.

Exercises (30 minutes)

1. Create namespace `sec`, ServiceAccount `reader`, a Role allowing `get, list` on pods and configmaps, and bind it to `reader`. Prove with `auth can-i` that `reader` can list pods but cannot delete them.

2. Create Pod hardened (busybox:1.36, sleep 3600) that runs as user 1000, group 2000, cannot escalate privileges, drops all capabilities, has a read-only root filesystem, and still has a writable /tmp. Show id from inside.
 3. Label namespace sec with pod-security.kubernetes.io/enforce=restricted. Try to run a plain nginx:1.27 Pod in it. Copy the error's advice into a working manifest.
 4. Deployment api in sec must run as ServiceAccount reader — change it with one command and confirm a Pod's spec.serviceAccountName.
 5. Create the backups.podkit.dev CRD above and a Backup named nightly; list it with the short name bk and inspect its schema with explain.
-

Day 2 solutions

Chapter 6

```
# 1
kubectl create deployment shop --image=nginx:1.25 --replicas=3
kubectl set image deployment/shop nginx=nginx:1.26 && kubectl annotate
deployment/shop kubernetes.io/change-cause="1.26"
kubectl set image deployment/shop nginx=nginx:1.27 && kubectl annotate
deployment/shop kubernetes.io/change-cause="1.27"
kubectl rollout history deployment/shop; kubectl rollout undo deployment/shop
--to-revision=2
# 2 – kubectl edit deploy shop: strategy.rollingUpdate: {maxSurge: 1,
maxUnavailable: 0}
# 3 – status: "Waiting for deployment "shop" rollout to finish: 1 out of 3 new
replicas have been updated..." then ProgressDeadlineExceeded;
# 3 old Pods keep serving (maxUnavailable 0); fix: kubectl rollout undo
deployment/shop
# 4
kubectl create ns bg
kubectl create deployment blue --image=nginx:1.26 -n bg $do | sed 's/app:
blue/app: front\n      version: blue/' ... # easier: create, then kubectl
edit both templates: labels app=front, version=blue|green
kubectl expose deployment blue --name=front --port=80 -n bg --
selector=app=front,version=blue
kubectl patch svc front -n bg -p '{"spec":{"selector":
{"app":"front","version":"green"}}}'
# 5
kubectl set resources deployment shop --requests=cpu=100m
kubectl autoscale deployment shop --min=2 --max=6 --cpu-percent=60
```

Chapter 7

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm install site bitnami/nginx -n helm-1 --create-namespace --set
replicaCount=2 --set service.type=ClusterIP
helm list -A
helm upgrade site bitnami/nginx -n helm-1 --set replicaCount=3 --set
service.type=ClusterIP
helm rollback site 1 -n helm-1; helm history site -n helm-1
helm template site bitnami/nginx --set replicaCount=2 > rendered.yaml; grep -n
'image:' rendered.yaml
# kustomize: base/kustomization.yaml (resources: [deployment.yaml]);
overlays/prod/kustomization.yaml:
# resources: [../../base] namespace: prod namePrefix: prod- replicas:
[{"name: hello, count: 3}] images: [{"name: nginx, newTag: "1.27"}]
kubectl create ns prod; kubectl apply -k overlays/prod; kubectl get deploy -n
prod -o wide
helm uninstall site -n helm-1; kubectl delete -k overlays/prod
```

Chapter 8

```
# 1 – readinessProbe: httpGet {path: /, port: 80}, initialDelaySeconds: 2,
periodSeconds: 5; livenessProbe: tcpSocket {port: 80}, periodSeconds: 10
kubectl describe pod probed | grep -E 'Liveness|Readiness'
# 2 – after 30 s the file is gone, the exec probe fails 3× (15 s), the
container restarts, the file reappears, and so on
# 3
```

```

kubectl create deployment crasher --image=busybox:1.36 -- sh -c 'echo boom;
exit 1'
kubectl logs deployment/crasher --previous > /tmp/reason.txt # or: kubectl
get pod -l app=crasher -o
jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated.reason}'
# 4 – apiVersion: apps/v1 and add spec.selector.matchLabels: {app: old}
# 5
kubectl run noshell --image=registry.k8s.io/pause:3.9
kubectl debug -it noshell --image=busybox:1.36 --target=noshell -- ps

```

Chapter 9

```

# 1
kubectl create ns sec; kubectl create sa reader -n sec
kubectl create role reader --verb=get,list --resource=pods,configmaps -n sec
kubectl create rolebinding reader --role=reader --serviceaccount=sec:reader -n
sec
kubectl auth can-i list pods -n sec --as=system:serviceaccount:sec:reader
# yes
kubectl auth can-i delete pods -n sec --as=system:serviceaccount:sec:reader
# no
# 2 – pod securityContext: runAsUser 1000, runAsGroup 2000; container:
allowPrivilegeEscalation false, readOnlyRootFilesystem true,
# capabilities.drop [ALL]; volumes: emptyDir "tmp" mounted at /tmp;
kubectl exec hardened -- id
# 3 – the rejection lists: allowPrivilegeEscalation != false, unrestricted
capabilities, runAsNonRoot != true, seccompProfile;
# add them to the container (runAsUser: 101 for nginx, or use
nginxinc/nginx-unprivileged)
# 4
kubectl set serviceaccount deployment api reader -n sec
kubectl get pod -n sec -l app=api -o
jsonpath='{.items[0].spec.serviceAccountName}'
# 5
kubectl apply -f crd.yaml; kubectl apply -f backup.yaml; kubectl get bk;
kubectl explain backup.spec

```

Day 3 — Network and rehearse

Chapter 10 — Services and Ingress

Plain words

Pods come and go and their IPs change. A **Service** is a stable name and IP in front of a group of Pods (picked by label). Inside the cluster, `web.apps.svc.cluster.local` always works, however many Pods there are. Kinds of Service:

Type	Reachable from	Use
ClusterIP (default)	inside the cluster only	app-to-app traffic
NodePort	every node's IP on a high port (30000–32767)	quick external access, kind/minikube
LoadBalancer	a cloud load balancer IP	production entry point per Service (costs money)
ExternalName	(a DNS alias to something outside)	<code>db.apps.svc → db.example.com</code>
headless (clusterIP: None)	returns Pod IPs directly	StatefulSets, client-side load balancing

An **Ingress** is one entry point (an HTTP router) that sends `shop.example.com/api` to one Service and `/` to another, with TLS — instead of one LoadBalancer per Service. It needs an **ingress controller** (nginx, Traefik, ...) running in the cluster; the Ingress object is only the routing table.

Step by step: Services

```
kubectl create deployment web --image=nginx:1.27 --replicas=2 --port=80
kubectl expose deployment web --port=8080 --target-port=80 #
ClusterIP: 8080 -> container 80
kubectl expose deployment web --name=web-np --port=80 --type=NodePort
kubectl get svc,endpoints,endpointslices
kubectl describe svc web # Endpoints:
10.244.x.y:80,10.244.x.z:80 <- must not be empty
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  type: ClusterIP
  selector: { app: web } # -> Pods with this label
  ports:
    - name: http
      port: 8080 # the Service's port
      targetPort: 80 # the container's port (number or port
NAME)
      protocol: TCP
      # nodePort: 30080 # only for NodePort/LoadBalancer;
optional (auto-assigned)
```

Test from inside the cluster (the exam's favourite check):

```
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -q0-
http://web:8080
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- nslookup
web.default.svc.cluster.local
kubectl port-forward svc/web 9090:8080      # then: curl localhost:9090 from
your machine
```

DNS names: SERVICE (same namespace), SERVICE.NAMESPACE, SERVICE.NAMESPACE.svc.cluster.local. Pods also get env vars like WEB_SERVICE_HOST if the Service existed before the Pod started.

Troubleshooting a Service that “does not work”:

1. `kubectl get endpoints web` — empty? $\Rightarrow$ selector $\neq$ Pod labels (compare `kubectl get svc web -o jsonpath='{.spec.selector}'` and `kubectl get pods --show-labels`), or Pods not Ready (readiness probe).
2. Ports: port is what clients use, `targetPort` must be the container’s real port.
3. From a test Pod, `wget -q0- http://POD_IP:80` works but `http://web:8080` does not $\Rightarrow$ Service config; both fail $\Rightarrow$ the app or a NetworkPolicy (Chapter 11).
4. `kubectl get svc -o wide` shows the selector; `kubectl describe svc` shows the endpoints.

Step by step: Ingress

```
kubectl create ingress shop --class=nginx \
  --rule="shop.example.com/=web:80" \
  --rule="shop.example.com/api=api:8080" \
  --rule="admin.example.com/*=admin:80,tls=admin-tls"
kubectl get ingress; kubectl describe ingress shop
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /      # controller-specific
behaviour
spec:
  ingressClassName: nginx                               # which controller
handles it
  defaultBackend:                                       # optional: anything that
matches no rule
    service: { name: web, port: { number: 80 } }
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls                             # kubectl create secret
tls shop-tls --cert --key
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix                             # Prefix | Exact |
ImplementationSpecific
            backend:
              service: { name: web, port: { number: 80 } }
          - path: /api
```

```

    pathType: Prefix
    backend:
      service: { name: api, port: { name: http } } # a named Service
port also works

```

pathType: Prefix matches /api and /api/anything; Exact matches only /api. A rule without host matches every hostname. The controller usually exposes itself as a NodePort or LoadBalancer Service in its own namespace (`kubectl get svc -n ingress-nginx`).

Test with a fake host header:

```
curl -H 'Host: shop.example.com' http://<node-ip>:<controller-nodeport>/api
```

Gateway API (Gateway, HTTPRoute) is the successor of Ingress and is now the standard for new work; know that it exists — the exam currently focuses on Ingress.

In podkit

- Every generated pod with `ports:` gets a ClusterIP Service with the same name; the in-cluster address is printed after deploy (`SERVICE_DNS=name.ns.svc.cluster.local` in `pod.env`). `service.type: NodePort|LoadBalancer` changes the type; `service.port` maps an external port to the first container port — exactly the `port/targetPort` pair above.
- The Java client reaches the server through its Service DNS name, and the server certificate’s SANs are the four forms of that name (`name`, `name.ns`, `name.ns.svc`, `name.ns.svc.cluster.local`).
- The SSO example uses Istio’s Gateway + VirtualService instead of an Ingress: one entry point, host-based routing (`web-alpha.DOMAIN → Service web-alpha:80`), TLS from a Secret — the same ideas, richer features (JWT checks, traffic splits). `podkit istio install` exposes the gateway as a NodePort Service (30080/30443) mapped to the laptop’s ports 80/443 by kind.

Exam tips

- `kubectl expose` takes the **Deployment/Pod name** and copies its labels into the selector — the fastest correct Service.
- “Expose on port 30007 of every node” ⇒ NodePort with an explicit `nodePort: 30007` (edit YAML).
- The test Pod: `kubectl run tmp --rm -it --image=busybox --restart=Never -- wget -q0- URL` (busybox has `wget` and `nslookup`, not `curl`; `curlimages/curl` has `curl`).
- Ingress: check `ingressClassName`; a missing or wrong class means no controller picks it up.
- Docs: *Service, Ingress, Debug Services*.

Exercises (25 minutes)

1. Create Deployment `api` (`nginx:1.27`, 3 replicas, container port 80) and expose it as ClusterIP Service `api` on port 5000. From a temporary busybox Pod, fetch the nginx welcome page through the Service.
2. Create a NodePort Service `api-ext` for the same Pods on node port 30500. Verify the port is listed in `kubectl get svc`.

3. Service broken selects app: `apii` (typo) — create it, show that it has no endpoints, and fix it with `kubectl patch`.
 4. Create an Ingress `site` (class `nginx`) routing `site.local/` to `api:5000` and `site.local/admin` to a Service `admin:80` (create a Pod+Service for it), with TLS secret `site-tls` (generate a self-signed cert with `openssl req -x509 -newkey rsa:2048 -nodes -keyout tls.key -out tls.crt -subj /CN=site.local -days 30`).
 5. Look at the Ingress with `describe`: which Service backs `/admin`, and what is the TLS host?
-

Chapter 11 — NetworkPolicy

Plain words

By default every Pod can talk to every other Pod, in every namespace. A **NetworkPolicy** is a firewall rule for Pods, written with labels: “Pods with label X accept traffic only from Pods with label Y on port 80, and may only talk to DNS and the database”. The cluster’s CNI (Calico, Cilium, kind’s default...) enforces it — with a CNI that does not support policies, the objects exist but do nothing (kind’s default CNI, kindnet, does **not** enforce them; the exam clusters do).

Rules of thumb:

- A policy **selects** Pods (podSelector) and then says what is allowed **in** (ingress) and/or **out** (egress). Everything not allowed by *some* policy is denied — but only for the directions listed in policyTypes.
- Policies are **additive** (allow-lists). There is no deny rule; you deny by not allowing.
- A Pod selected by no policy accepts everything.

Step by step: default deny, then allow

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: apps }
spec:
  podSelector: {} # every Pod in the namespace
  policyTypes: [Ingress, Egress] # deny both directions (nothing is allowed
below)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: web-allow, namespace: apps }
spec:
  podSelector:
    matchLabels: { app: web } # the Pods this policy protects
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - podSelector: # same namespace, label front=true
        matchLabels: { role: frontend }
      - namespaceSelector: # OR: any Pod in a namespace labelled
        matchLabels: { team: ops }
        team=ops
      - namespaceSelector: # AND (one list item with two selectors):
        matchLabels: { team: qa } # role=tester IN namespaces labelled
        Pods labelled
        team=qa
      podSelector:
        matchLabels: { role: tester }
    - ipBlock:
        cidr: 10.0.0.0/16
        except: [10.0.5.0/24]
  ports:
    - protocol: TCP
      port: 80 # number or port name; omit `ports` = all
ports
egress:
```

```

- to:                                     # DNS: every policy with Egress needs this or
name lookups break
  - namespaceSelector:
      matchLabels: { kubernetes.io/metadata.name: kube-system }
  ports:
    - { protocol: UDP, port: 53 }
    - { protocol: TCP, port: 53 }
- to:
  - podSelector:
      matchLabels: { app: db }
  ports:
    - { protocol: TCP, port: 5432 }

```

The two most important shapes to recognise:

```

# one item, two selectors => AND (Pods matching BOTH, in matching namespaces)
- from:
  - namespaceSelector: { matchLabels: { team: qa } }
  - podSelector:       { matchLabels: { role: tester } }
# two items               => OR (either)
- from:
  - namespaceSelector: { matchLabels: { team: qa } }
  - podSelector:       { matchLabels: { role: tester } }

```

Every namespace carries the automatic label `kubernetes.io/metadata.name: <name>` — use it in namespaceSelectors instead of inventing labels.

```

kubectl get networkpolicy -n apps; kubectl describe netpol web-allow -n apps
kubectl label namespace ops team=ops                                     # give the source
namespace the label the policy expects
kubectl run tester --rm -it --image=busybox:1.36 --restart=Never -n apps -l
role=frontend -- wget -q0- --timeout=3 http://web

```

Testing: run a test Pod **with the labels the policy expects** in the right namespace; a wget timeout means denied.

In podkit

- `networkPolicy.enabled: true` in a PodConfig generates a default-deny policy for the pod plus explicit allows: DNS to kube-system, HTTPS to the world when a cloud identity is used, ingressFrom Pods (the Java server allows only the client), egressTo Pods, and egress CIDRs (the Postgres example allows `10.50.0.0/16:5432`). Open `build/aws/pg-client/k8s/workload.yaml` and find every shape from this chapter in one object — `podgen/k8s.py — network_policy()`.
- The Terraform pod module renders the same policy with dynamic "egress" blocks — compare them.
- In the Istio example the sidecar's mTLS + AuthorizationPolicy do a similar job at L7 (HTTP identity), while NetworkPolicy works at L3/L4 (IPs and ports). Real clusters use both.

Exam tips

- Start with the selector: *which* Pods does the task protect? That is `spec.podSelector`.
- "Only allow from X" ⇒ you also need the deny: either `policyTypes` on a policy that lists X only (anything not listed is denied for that direction), or a separate default-deny.

- Forgetting DNS egress is the #1 self-inflicted wound: add the kube-system/53 rule whenever Egress is in policyTypes.
- An empty podSelector: {} matches all Pods; an empty ingress: [{}] allows everything; ingress: [] (or absent with Ingress in policyTypes) allows nothing.
- Docs: *Network Policies* — the sample at the top of the page has every field; copy it.

Exercises (25 minutes)

1. In namespace np, create Pods web (nginx:1.27, label app=web) and client (busybox:1.36, label app=client, sleep 3600) and a Service web. Apply a default-deny ingress policy for the namespace. Show that client can no longer reach web.
2. Allow ingress to web on TCP 80 only from Pods labelled app=client. Test from client (should work) and from a Pod without the label (should time out).
3. Create namespace ext labelled team=ext with a busybox Pod. Extend the policy so Pods from namespace ext may also reach web, and test it.
4. Add an egress policy for client: it may only talk to web on 80 and to DNS. Verify nslookup web still works and that wget http://kubernetes.default times out.
5. Read build/local/web-alpha/k8s/workload.yaml after generating the SSO example with networkPolicy.enabled: true (add the block and regenerate). Which rule would you have to add so the Istio sidecar can reach istiod on port 15012 in istio-system?

Chapter 12 — Exam day: strategy, a full mock exam, and a speed drill

The plan for two hours

1. **First 2 minutes.** Set aliases (`$do`, `$now`), `:set` paste knowledge, open `kubernetes.io/docs` in the second tab and search “Deployment” once so the search works.
2. **Pass 1 (≈ 75 min).** Do every task you know cold. Skip anything that smells expensive (custom Ingress controllers, weird CRDs) — write its number down.
3. **Pass 2 (≈ 35 min).** The skipped ones, hardest last.
4. **Last 10 minutes.** Check: every task’s **context** was set, every object is in the **right namespace** with the **exact name**, Pods are Running (`kubectl get pods -A --field-selector status.phase!=Running`).

Scoring facts that change behaviour:

- Partial credit exists inside a task (each check is scored) — an 80 %-done task is worth something; a perfect task you never reached is worth nothing.
- Nobody sees *how* you did it: imperative commands, `kubectl edit`, copied docs — all fine.
- Rerunning `kubectl apply` is free; deleting the wrong thing is not (`--dry-run=client` first).

Habits:

Do	Don't
<code>kubectl config use-context</code> per task	assume the context carried over
<code>kubectl get ... -n NS</code> to verify before moving on	trust that <code>apply</code> silently worked
copy YAML from the docs, then edit	type YAML from memory
use <code>explain</code> for field names	guess spelling (<code>initialDelaySecond</code>)
<code>kubectl replace --force -f</code> to change immutable Pod fields	fight <code>kubectl edit</code> errors
read the Events in <code>describe</code>	stare at YAML for ten minutes

Mock exam — 2 hours, 18 tasks

Use a fresh cluster (`podkit kind destroy && podkit kind create`). Create the namespaces first: `for n in mock-a mock-b mock-c mock-d; do kubectl create ns $n; done`. Set a timer. Suggested minutes per task are in brackets. Solutions follow the list — do not peek.

1. **[4] Pod basics.** In `mock-a`, create Pod `alpha` with image `busybox:1.36` running `sh -c 'while true; do date; sleep 5; done'`, environment variable `TEAM=blue`, CPU request `50m`, memory limit `64Mi`. Save its YAML to `/tmp/alpha.yaml`.

2. **[5] Job.** In mock-a, create Job once with image busybox:1.36 that prints done, runs 4 completions with 2 in parallel, retries at most once, and is deleted 100 s after it finishes.
3. **[4] CronJob.** In mock-a, CronJob beat runs date with busybox:1.36 every Monday at 03:30, forbids concurrent runs and keeps 2 successful Jobs. Then start one run immediately as Job beat -now.
4. **[6] Multi-container.** In mock-b, Pod combo has an init container that writes ready into /shared/state (emptyDir), a main container app (nginx:1.27) serving /shared at /usr/share/nginx/html, and a sidecar tailer (busybox:1.36) running tail -F /shared/state that starts before app.
5. **[5] ConfigMap + Secret.** In mock-b, create ConfigMap cfg (MODE=fast, LEVEL=3) and Secret key (API_KEY=xyz). Pod consumer (busybox:1.36, sleep 3600) must expose every ConfigMap key as env vars and mount the Secret as files under /etc/key.
6. **[5] Volumes.** In mock-b, create a PVC store (ReadWriteOnce, 200Mi, default class) and Pod keeper (busybox:1.36, sleep 3600) mounting it at /data. Write hi to /data/f and confirm the PVC is Bound.
7. **[5] Deployment rollout.** In mock-c, create Deployment roll (nginx:1.25, 3 replicas), update to nginx:1.26 recording the cause “bump”, then roll back to the first revision. Configure the strategy so at most 1 Pod is unavailable and no extra Pods are created.
8. **[6] Blue/green.** In mock-c, Deployments v1 (nginx:1.26) and v2 (nginx:1.27), both with Pod label app=site plus ver=v1/ver=v2. Service site (port 80) must send all traffic to v2.
9. **[5] Helm.** Install chart bitnami/nginx as release hm in namespace mock-c with replicaCount=2, then upgrade to replicaCount=3, then roll back to revision 1. Write the current revision number to /tmp/rev.txt.
10. **[4] Kustomize.** Directory /tmp/kz contains a Deployment kz-web (nginx:1.26) and a kustomization.yaml. Add an overlay /tmp/kz/staging that sets namespace mock-c, prefix stg- and 2 replicas; apply it.
11. **[5] Probes.** In mock-d, Deployment healthy (nginx:1.27, 2 replicas) needs a readiness probe GET / on port 80 every 3 s starting after 5 s, and a liveness probe on TCP 80 every 10 s with 5 failures allowed.
12. **[5] Debug.** Pod sick in mock-d (create it from busybox:1.36 with command sh -c 'exit 3') keeps crashing. Write the container's exit code to /tmp/exit.txt and the reason string (from lastState) to /tmp/why.txt, then fix the Pod so it stays running (sleep 3600).
13. **[4] Deprecated API.** Fix /tmp/old-ingress.yaml (an extensions/v1beta1 Ingress routing / to Service web port 80 for host old.local) so it applies in mock-d with class nginx.
14. **[6] RBAC.** In mock-d, ServiceAccount deployer must be able to create, get, list and update Deployments and get/list Pods — nothing else. Prove it with auth can-i for create deployments (yes) and delete pods (no).
15. **[5] SecurityContext.** In mock-d, Pod locked (busybox:1.36, sleep 3600) runs as user 1001, non-root, no privilege escalation, drops all capabilities, adds NET_BIND_SERVICE, read-only root filesystem with a writable emptyDir at /tmp.

16. **[5] Service + Ingress.** In mock-a, create Deployment front (nginx:1.27, port 80) and a NodePort Service front on node port 30100; then an Ingress front (class nginx) for host front.local, path / → front:80.
17. **[6] NetworkPolicy.** In mock-b, Pods labelled app=db accept ingress only from Pods labelled app=api in the same namespace on TCP 5432, and only from namespace mock-c (any Pod) on TCP 9000. Everything else is denied. Egress is not restricted.
18. **[5] CRD.** A CRD widgets.shop.io exists (create it from the snippet in Chapter 9 with plural widgets, kind Widget, fields spec.size string and spec.count integer). Create Widget w1 in mock-d with size L and count 2, and list Widgets with the short name wd.

Mock exam solutions

```
# 1
kubectl run alpha -n mock-a --image=busybox:1.36 --env=TEAM=blue \
  $do -- sh -c 'while true; do date; sleep 5; done' > /tmp/alpha.yaml
# add under the container: resources: {requests: {cpu: 50m}, limits:
{memory: 64Mi}}
kubectl apply -f /tmp/alpha.yaml

# 2
kubectl create job once -n mock-a --image=busybox:1.36 $do -- echo done >
/tmp/job.yaml
# spec: completions: 4, parallelism: 2, backoffLimit: 1,
ttlSecondsAfterFinished: 100
kubectl apply -f /tmp/job.yaml

# 3
kubectl create cronjob beat -n mock-a --image=busybox:1.36 --schedule="30 3 * *
1" $do -- date > /tmp/cj.yaml
# spec: concurrencyPolicy: Forbid, successfulJobsHistoryLimit: 2
kubectl apply -f /tmp/cj.yaml; kubectl create job beat-now -n mock-a --
from=cronjob/beat

# 4 (Pod YAML) volumes: [{name: shared, emptyDir: {}}]
#   initContainers:
#     - {name: init, image: busybox:1.36, command: [sh, -c, "echo ready >
/shared/state"], volumeMounts: [{name: shared, mountPath: /shared}]}
#     - {name: tailer, image: busybox:1.36, restartPolicy: Always, command:
[sh, -c, "tail -F /shared/state"], volumeMounts: [{name: shared, mountPath:
/shared}]}
#   containers:
#     - {name: app, image: nginx:1.27, volumeMounts: [{name: shared, mountPath:
/usr/share/nginx/html}]}

# 5
kubectl create cm cfg -n mock-b --from-literal=MODE=fast --from-literal=LEVEL=3
kubectl create secret generic key -n mock-b --from-literal=API_KEY=xyz
kubectl run consumer -n mock-b --image=busybox:1.36 $do -- sleep 3600 >
/tmp/c.yaml
# envFrom: [{configMapRef: {name: cfg}}]; volumes: [{name: key, secret:
{secretName: key}}]; volumeMounts: [{name: key, mountPath: /etc/key}]
# 6 PVC: accessModes [ReadWriteOnce], resources.requests.storage 200Mi (no
storageClassName -> default)
#   Pod: volumes: [{name: data, persistentVolumeClaim: {claimName: store}}],
mount /data
kubectl exec keeper -n mock-b -- sh -c 'echo hi > /data/f'; kubectl get pvc
store -n mock-b

# 7
kubectl create deployment roll -n mock-c --image=nginx:1.25 --replicas=3
kubectl set image deployment/roll nginx=nginx:1.26 -n mock-c
```

```

kubectl annotate deployment/roll -n mock-c kubernetes.io/change-cause=bump
kubectl rollout undo deployment/roll -n mock-c --to-revision=1
kubectl patch deployment roll -n mock-c -p '{"spec":{"strategy":{"rollingUpdate":{"maxUnavailable":1,"maxSurge":0}}}}'
# 8
kubectl create deployment v1 -n mock-c --image=nginx:1.26 $do > /tmp/v1.yaml
# template labels: app: site, ver: v1 (and selector matchLabels the same)
kubectl create deployment v2 -n mock-c --image=nginx:1.27 $do > /tmp/v2.yaml
# app: site, ver: v2
kubectl apply -f /tmp/v1.yaml -f /tmp/v2.yaml
kubectl create service clusterip site -n mock-c --tcp=80:80 $do > /tmp/svc.yaml
# selector: {app: site, ver: v2}
kubectl apply -f /tmp/svc.yaml; kubectl get endpoints site -n mock-c
# 9
helm repo add bitnami https://charts.bitnami.com/bitnami; helm repo update
helm install hm bitnami/nginx -n mock-c --set replicaCount=2
helm upgrade hm bitnami/nginx -n mock-c --set replicaCount=3
helm rollback hm 1 -n mock-c
helm history hm -n mock-c | tail -1 | awk '{print $1}' > /tmp/rev.txt # 3
# 10 /tmp/kz/staging/kustomization.yaml:
# resources: [../] namespace: mock-c namePrefix: stg- replicas: [{name:
kz-web, count: 2}]
kubectl apply -k /tmp/kz/staging
# 11
kubectl create deployment healthy -n mock-d --image=nginx:1.27 --replicas=2 $do
> /tmp/h.yaml
# readinessProbe: {httpGet: {path: /, port: 80}, initialDelaySeconds: 5,
periodSeconds: 3}
# livenessProbe: {tcpSocket: {port: 80}, periodSeconds: 10, failureThreshold:
5}
# 12
kubectl get pod sick -n mock-d -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}' >
/tmp/exit.txt
kubectl get pod sick -n mock-d -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}' >
/tmp/why.txt
kubectl get pod sick -n mock-d -o yaml > /tmp/sick.yaml # change command to
sleep 3600
kubectl replace --force -f /tmp/sick.yaml
# 13 apiVersion: networking.k8s.io/v1; spec.ingressClassName: nginx;
rules[0].http.paths[0]: {path: /, pathType: Prefix, backend: {service: {name:
web, port: {number: 80}}}}
kubectl apply -f /tmp/old-ingress.yaml -n mock-d
# 14
kubectl create sa deployer -n mock-d
kubectl create role deployer -n mock-d --verb=create,get,list,update --
resource=deployments
kubectl create role pod-view -n mock-d --verb=get,list --resource=pods
kubectl create rolebinding deployer -n mock-d --role=deployer --
serviceaccount=mock-d:deployer
kubectl create rolebinding pod-view -n mock-d --role=pod-view --
serviceaccount=mock-d:deployer
kubectl auth can-i create deployments -n mock-d --
as=system:serviceaccount:mock-d:deployer # yes
kubectl auth can-i delete pods -n mock-d --as=system:serviceaccount:mock-
d:deployer # no
# 15 securityContext (pod): runAsUser 1001, runAsNonRoot true; (container):

```

```

allowPrivilegeEscalation false,
#   readOnlyRootFilesystem true, capabilities {drop: [ALL], add:
[NET_BIND_SERVICE]}; emptyDir at /tmp
# 16
kubectl create deployment front -n mock-a --image=nginx:1.27 --port=80
kubectl expose deployment front -n mock-a --type=NodePort --port=80 $do >
/tmp/s.yaml # add nodePort: 30100
kubectl apply -f /tmp/s.yaml
kubectl create ingress front -n mock-a --class=nginx --
rule="front.local/=front:80"
# 17 (NetworkPolicy in mock-b) podSelector {app: db}; policyTypes [Ingress];
#   ingress:
#     - from: [{podSelector: {matchLabels: {app: api}}}]
ports: [{protocol: TCP, port: 5432}]
#     - from: [{namespaceSelector: {matchLabels: {kubernetes.io/metadata.name:
mock-c}}}] ports: [{protocol: TCP, port: 9000}]
# 18 CRD names: {plural: widgets, singular: widget, kind: Widget, shortNames:
[wd]}; group shop.io
kubectl apply -f /tmp/widget.yaml # apiVersion: shop.io/v1, kind: Widget,
metadata: {name: w1, namespace: mock-d}, spec: {size: L, count: 2}
kubectl get wd -n mock-d

```

Score yourself: each task is either right (all checks) or partly right. Under 12 of 18 fully right ⇒ redo the weak chapters tonight and run the drill below twice tomorrow morning.

Speed drill — 20 one-liners in 15 minutes

Type each from memory; check the answer only after you tried.

1. A Pod `t` from `nginx:1.27` in namespace `x`, port 80, label `app=t`.
2. Its YAML to a file without creating it.
3. A Deployment `d`, 3 replicas, `httpd:2.4`, exposed on 8080 — 80 as ClusterIP.
4. Scale `d` to 5.
5. Change `d`'s image to `httpd:2.4.59` and wait for it.
6. Undo that.
7. The container names of `d`.
8. A ConfigMap from `app.env`.
9. A Secret with `pw=123`, then decode it.
10. Which ServiceAccount can list Secrets in `x`? (test `sa1`)
11. All Pods not Running, cluster-wide.
12. Last 20 events in `x`, newest last.
13. The image of every Pod in `x`, one per line.
14. A Job `j` running `echo hi` with `busybox:1.36`, then its logs.
15. A CronJob every hour at minute 15.
16. Add label `tier=web` to all Pods with `app=t`.
17. A temporary busybox Pod that fetches `http://d:8080` and disappears.
18. Delete Pod `t` immediately.
19. Show the fields of `pod.spec.securityContext`.
20. Node port of Service `d` after making it NodePort.

```

1 kubectl run t -n x --image=nginx:1.27 --port=80 -l app=t
2 kubectl run t -n x --image=nginx:1.27 --port=80 -l app=t $do > t.yaml
3 kubectl create deployment d --image=httpd:2.4 --replicas=3 && kubectl expose
deployment d --port=8080 --target-port=80
4 kubectl scale deployment d --replicas=5
5 kubectl set image deployment/d httpd=httpd:2.4.59 && kubectl rollout status
deployment/d
6 kubectl rollout undo deployment/d
7 kubectl get deployment d -o
jsonpath='{.spec.template.spec.containers[*].name}'
8 kubectl create configmap app --from-env-file=app.env
9 kubectl create secret generic s --from-literal=pw=123 && kubectl get secret
s -o jsonpath='{.data.pw}' | base64 -d
10 kubectl auth can-i list secrets -n x --as=system:serviceaccount:x:sa1
11 kubectl get pods -A --field-selector status.phase!=Running
12 kubectl get events -n x --sort-by=.lastTimestamp | tail -20
13 kubectl get pods -n x -o jsonpath='{range .items[*]}
{.spec.containers[0].image}{ "\n" }{end}'
14 kubectl create job j --image=busybox:1.36 -- echo hi && kubectl logs job/j
15 kubectl create cronjob c --image=busybox:1.36 --schedule="15 * * * *" --
date
16 kubectl label pods -l app=t tier=web
17 kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -q0-
http://d:8080
18 kubectl delete pod t --force --grace-period=0
19 kubectl explain pod.spec.securityContext
20 kubectl patch svc d -p '{"spec":{"type":"NodePort"}}' && kubectl get svc d -
o jsonpath='{.spec.ports[0].nodePort}'

```

Day 3 solutions (Chapters 10–11)

Chapter 10

```

# 1
kubectl create deployment api --image=nginx:1.27 --replicas=3 --port=80
kubectl expose deployment api --port=5000 --target-port=80
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -q0-
http://api:5000 | head -3
# 2
kubectl expose deployment api --name=api-ext --type=NodePort --port=80 $do >
np.yaml # add nodePort: 30500 under ports[0]
kubectl apply -f np.yaml; kubectl get svc api-ext
# 3
kubectl expose deployment api --name=broken --port=80 --selector=app=apii
kubectl get endpoints broken # <none>
kubectl patch svc broken -p '{"spec":{"selector":{"app":"apii"}}}'
# 4
openssl req -x509 -newkey rsa:2048 -nodes -keyout tls.key -out tls.crt -subj
/CN=site.local -days 30
kubectl create secret tls site-tls --cert=tls.crt --key=tls.key
kubectl run admin --image=nginx:1.27 --port=80 -l app=admin && kubectl expose
pod admin --port=80
kubectl create ingress site --class=nginx --rule="site.local/=api:5000" --
rule="site.local/admin=admin:80,tls=site-tls"
# 5
kubectl describe ingress site # Rules: /admin -> admin:80; TLS: site-tls
terminates site.local

```

Chapter 11

```
# 1
kubectl create ns np
kubectl run web -n np --image=nginx:1.27 -l app=web --port=80 && kubectl expose
pod web -n np --port=80
kubectl run client -n np --image=busybox:1.36 -l app=client -- sleep 3600
# default-deny: podSelector {}, policyTypes [Ingress]
kubectl exec client -n np -- wget -q0- --timeout=3 http://web # times out
# 2 – policy: podSelector {app: web}; ingress from podSelector {app: client};
ports TCP 80
kubectl exec client -n np -- wget -q0- --timeout=3 http://web # works
kubectl run other -n np --rm -it --image=busybox:1.36 --restart=Never -- wget -
q0- --timeout=3 http://web # times out
# 3
kubectl create ns ext; kubectl label ns ext team=ext
kubectl run extc -n ext --image=busybox:1.36 -- sleep 3600
# add a second `from` item: namespaceSelector {matchLabels: {team: ext}}
kubectl exec extc -n ext -- wget -q0- --timeout=3 http://web.np # works
# 4 – policy: podSelector {app: client}; policyTypes [Egress]; egress: to
podSelector {app: web} port 80; to namespaceSelector
# {kubernetes.io/metadata.name: kube-system} ports 53 UDP/TCP
kubectl exec client -n np -- nslookup web # works
kubectl exec client -n np -- wget -q0- --timeout=3 https://kubernetes.default
# times out
# 5 – an egress rule: to namespaceSelector {kubernetes.io/metadata.name: istio-
system}, ports TCP 15012
# (podkit's generated policy allows DNS + declared CIDRs/pods only; Istio
sidecars also need 15012 to istiod)
```

Appendix A — kubectl cheat sheet (print this)

```
# ---- setup ----
alias k=kubectl; export do="--dry-run=client -o yaml"; export now="--force --
grace-period=0"
kubectl config use-context CTX; kubectl config set-context --current --
namespace=NS

# ---- create (imperative) ----
kubectl run POD --image=IMG [--port=80] [-l k=v] [--env=K=V] [--restart=Never|
OnFailure] [-- cmd args]
kubectl create deployment D --image=IMG --replicas=N [--port=80]
kubectl create job J --image=IMG -- cmd; kubectl create job J --
from=cronjob/C
kubectl create cronjob C --image=IMG --schedule="*/5 * * * *" -- cmd
kubectl create configmap CM --from-literal=K=V --from-file=F --from-env-file=E
kubectl create secret generic S --from-literal=K=V; kubectl create secret tls
S --cert=c --key=k
kubectl create secret docker-registry R --docker-server= --docker-username= --
docker-password=
kubectl create serviceaccount SA; kubectl create namespace NS; kubectl create
quota Q --hard=pods=5
kubectl create role R --verb=get,list --resource=pods; kubectl create
rolebinding RB --role=R --serviceaccount=NS:SA
kubectl create clusterrole CR --verb=get --resource=nodes; kubectl create
clusterrolebinding CRB --clusterrole=CR --user=U
kubectl expose deployment D --port=80 --target-port=8080 [--type=NodePort] [--
name=S]
kubectl create ingress I --class=nginx --rule="host/path=svc:port[,tls=secret]"
kubectl autoscale deployment D --min=2 --max=5 --cpu-percent=70

# ---- change ----
kubectl set image deployment/D CONTAINER=IMG:TAG; kubectl set env deployment/D
K=V
kubectl set resources deployment/D -c C --requests=cpu=100m,memory=64Mi --
limits=cpu=200m,memory=128Mi
kubectl set serviceaccount deployment/D SA; kubectl scale deployment D --
replicas=N
kubectl edit KIND NAME; kubectl patch KIND NAME -p '{"spec":{"...}}'; kubectl
label KIND NAME k=v [--overwrite]
kubectl annotate deployment/D kubernetes.io/change-cause="why"
kubectl rollout status|history|undo [--to-revision=N]|restart|pause|resume
deployment/D
kubectl replace --force -f f.yaml # recreate (immutable Pod fields)
kubectl apply -f f.yaml; kubectl apply -k DIR; kubectl delete -f f.yaml;
kubectl delete pod P $now

# ---- read ----
kubectl get KIND [-n NS|-A] [-o wide|yaml|json|name] [--show-labels] [-l k=v]
[--field-selector k=v] [-w]
kubectl get pods -o jsonpath='{.items[*].metadata.name}'; -o custom-
columns=NAME:.metadata.name,IMG:.spec.containers[0].image
kubectl describe KIND NAME; kubectl get events -n NS --sort-by=.lastTimestamp
kubectl logs POD [-c C] [-f] [--previous] [--tail=N] [--since=5m]; kubectl
logs deployment/D; kubectl logs -l k=v --all-containers
kubectl exec -it POD [-c C] -- sh; kubectl cp POD:/path ./local; kubectl
port-forward svc/S 8080:80
kubectl top pod [--containers]; kubectl top node
kubectl debug -it POD --image=busybox:1.36 --target=C; kubectl debug node/N -
```

```
it --image=busybox:1.36
kubectl auth can-i VERB RESOURCE [-n NS] [--as=system:serviceaccount:NS:SA] [--list]
kubectl explain KIND.spec.field [--recursive]; kubectl api-resources [--namespaced=false]; kubectl api-versions
kubectl get endpoints S; kubectl get pvc,pv,sc; kubectl get crd; kubectl get netpol

# ---- helm ----
helm repo add NAME URL; helm repo update; helm search repo X [--versions]; helm show values CHART
helm install R CHART -n NS --create-namespace [--set k=v] [-f values.yaml] [--version V]
helm upgrade [--install] R CHART -n NS ...; helm rollback R REV -n NS; helm history R -n NS; helm list -A
helm uninstall R -n NS; helm template R CHART; helm get values R -n NS; helm get manifest R -n NS

# ---- docker / podman ----
docker build -t IMG:TAG [-f Dockerfile] DIR; docker images; docker tag A B;
docker push B
docker save -o file.tar IMG:TAG; docker load -i file.tar; docker run --rm -p 8080:80 IMG:TAG
```

Appendix B — YAML skeletons you may need to type

Pod with everything common

```
apiVersion: v1
kind: Pod
metadata: { name: app, namespace: ns, labels: { app: app } }
spec:
  serviceAccountName: app-sa
  securityContext: { runAsUser: 1000, runAsNonRoot: true, fsGroup: 2000 }
  initContainers:
    - { name: init, image: busybox:1.36, command: [sh, -c, "echo init"] }
  containers:
    - name: app
      image: nginx:1.27
      ports: [{ containerPort: 80, name: http }]
      command: ["nginx"]
      args: ["-g", "daemon off;"]
      env:
        - { name: A, value: "1" }
        - { name: B, valueFrom: { configMapKeyRef: { name: cm, key: b } } }
        - { name: C, valueFrom: { secretKeyRef: { name: s, key: c } } }
        - { name: NODE, valueFrom: { fieldRef: { fieldPath: spec.nodeName } } }
      envFrom: [{ configMapRef: { name: cm } }, { secretRef: { name: s } }]
      resources: { requests: { cpu: 100m, memory: 64Mi }, limits: { cpu: 200m,
memory: 128Mi } }
      readinessProbe: { httpGet: { path: /, port: http }, periodSeconds: 5 }
      livenessProbe: { tcpSocket: { port: 80 }, initialDelaySeconds: 10,
periodSeconds: 10 }
      securityContext: { allowPrivilegeEscalation: false,
readOnlyRootFilesystem: true, capabilities: { drop: [ALL] } }
      volumeMounts:
        - { name: data, mountPath: /data }
        - { name: cfg, mountPath: /etc/app/app.conf, subPath: app.conf }
        - { name: tmp, mountPath: /tmp }
      volumes:
        - { name: data, persistentVolumeClaim: { claimName: data } }
        - { name: cfg, configMap: { name: cm } }
        - { name: tmp, emptyDir: {} }
      nodeSelector: { disktype: ssd }
      tolerations: [{ key: dedicated, operator: Equal, value: batch, effect:
NoSchedule }]
      restartPolicy: Always
```

Deployment (strategy + selector) — see Chapter 6. **Job / CronJob** — Chapter 2. **Service / Ingress** — Chapter 10. **NetworkPolicy** — Chapter 11. **Role / RoleBinding** — Chapter 9. **PVC / LimitRange / ResourceQuota** — Chapter 5. **HPA** — Chapter 6.

Where in the docs (search box terms): Pod, Deployment, Job, CronJob, ConfigMap, Secret, Persistent Volumes, Liveness Readiness Startup, Service, Ingress, Network Policies, RBAC, Security Context, Pod Security Standards, Custom Resources, Manage Resources Containers, Kustomize, Deprecated API Migration Guide.

Appendix C — Where each exam topic lives in podkit

Exam topic	podkit file / command
Deployment with probes, resources, labels, strategy	podgen/k8s.py — deployment(), any build/*/*/k8s/workload.yaml
Container images, Dockerfile, USER, CMD	command-pod/Dockerfile, examples/java-mtls/Dockerfile, podkit image build
Multi-container / sidecars	Istio sidecars in examples/sso-istio-keycloak/, command pod emptyDir
ConfigMap files with subPath	PodConfig files: — ConfigMap <name>-files
Secrets from env / files; cloud secret sync	podkit secret sync, scripts/lib/cloud.sh, tls keystore Secrets
Requests/limits defaults	schema/pod-config.schema.json (resources), terraform/modules/pod/variables.tf
Rolling update knobs, checksum-triggered restarts	podgen/k8s.py (maxSurge/maxUnavailable, podkit.dev/*-checksum)
HPA (autoscaling/v2)	podgen/k8s.py — hpa()
Helm install/upgrade/uninstall	scripts/resources/istio.sh
Generator ≈ Kustomize overlays	podgen/, podkit generate --provider ...
Probes (http/tcp)	PodConfig probe:: Java server uses tcp
Debugging order	podkit pod status, podkit pod logs, podkit doctor
ServiceAccount per pod, automountServiceAccountToken: false	podgen/k8s.py — service_account()
Namespaced RBAC	command-pod/k8s/rbac.yaml, podkit command-pod grant
SecurityContext (restricted profile)	podgen/k8s.py — deployment() container/pod security
Pod Security Admission labels	podkit namespace create NS --pss restricted
CRDs / operators	Istio CRDs in manifests/sso/*.yaml
Services, DNS names, ports	service_dns / SERVICE_PORT in pod.env; service: in PodConfig
Ingress-like routing with TLS	manifests/sso/gateway.yaml, manifests/sso/app.yaml (Istio)

Exam topic	podkit file / command
NetworkPolicy (all shapes)	podgen/k8s.py -- network_policy(); examples/pg-client.yaml
Jobs / one-shot work	ansible/playbooks/run-script.yml pattern; configure steps
Certificates (TLS/mTLS)	podkit certs, podkit tls, configure/java- keystore.sh

Appendix D — Glossary

Term	In one line
Admission controller	code that checks or changes an object after authorization, before it is stored
Affinity / anti-affinity	rules that attract Pods to (or away from) nodes or other Pods
Annotation	non-identifying metadata (key: value) tools read; not for selection
API group	family of kinds: core (""), apps, batch, networking.k8s.io, rbac.authorization.k8s.io
CNI	the network plugin; decides whether NetworkPolicies are enforced
ConfigMap	non-secret settings as key/value pairs or files
Container runtime	containerd/CRI-O; runs the containers on a node
CRD	Custom Resource Definition: adds a new kind to the API
DaemonSet	one Pod on every node
Deployment	keeps N Pods, rolls out changes, keeps history
emptyDir	scratch volume that lives as long as the Pod
Endpoints / EndpointSlice	the Pod IPs behind a Service
etcd	the control plane's database
HPA	Horizontal Pod Autoscaler: scales replicas on metrics
Ingress	HTTP(S) routing rules handled by an ingress controller
Init container	runs to completion before the app containers
Job / CronJob	run-to-completion Pods, once or on a schedule
kubelet	the node agent that starts Pods
Label / selector	key/value stickers and the queries that match them
LimitRange	per-container defaults and bounds in a

Term	In one line
	namespace
Liveness / readiness / startup probe	restart / take-traffic / has-started health checks
Namespace	logical folder for objects
NetworkPolicy	allow-list firewall for Pods by labels/ports
Node	a machine in the cluster
Operator	a controller that manages custom resources
PV / PVC / StorageClass	a disk / a request for one / how to create one automatically
QoS class	Guaranteed / Burstable / BestEffort, from requests and limits
RBAC	Role-Based Access Control: Roles + Bindings
ReplicaSet	keeps N identical Pods; owned by a Deployment
Requests / limits	reserved / maximum CPU and memory per container
ResourceQuota	totals per namespace
Rolling update	replace Pods gradually; maxSurge, maxUnavailable
Secret	sensitive key/value data, base64 in the API
SecurityContext	user, group, capabilities, read-only FS, privilege settings
Service	stable virtual IP + DNS name for a set of Pods
ServiceAccount	identity of a Pod towards the API
Sidecar	helper container in the same Pod (native: init container with restartPolicy: Always)
StatefulSet	ordered, named Pods with their own disks
Taint / toleration	node repels Pods unless they tolerate the taint
Volume	storage attached to a Pod (emptyDir, configMap, secret, PVC, hostPath...)

Appendix E — After the book

- **Practice more:** the `killer.sh` CKAD simulator (two sessions come with the exam registration) — harder than the exam, with the same look and feel.
- **Docs to bookmark on exam day:** the `kubectl` cheat sheet page, *Deployments*, *Network Policies*, *Configure Liveness, Readiness and Startup Probes*, *RBAC*, *Ingress*.
- **Keep building with podkit:** every time you add a feature (a Job step, a sidecar, a NetworkPolicy rule), open the generated YAML — it is the best CKAD flash card there is, because it is real.

Good luck. Set the context, read the task twice, verify, move on.